

UNIVERSITÀ DEGLI STUDI DI VERONA

CORSO DI LAUREA IN INFORMATICA

Algoritmi

Federico Brutti
federico.brutti@studenti.univr.it

*Non ho letteralmente nulla da dire. Miglior professore della mia intera
carriera scolastica e universitaria. - L'autore*

Indice

1	Fondamenti	4
1.1	Complessità e notazione asintotica	4
1.2	Equazioni di ricorrenza	8
1.3	Teorema dell'esperto	11
1.4	Esercizi svolti	13
2	Ordinamento e selezione	14
2.1	Problema dell'ordinamento	14
2.2	Insertion Sort	14
2.3	Merge Sort	15
2.4	Struttura Heap e Heap Sort	18
2.5	Quick Sort classico e probabilistico	21
2.6	Algoritmi di ordinamento lineari e Counting Sort	24
2.7	Radix Sort	26
2.8	Bucket Sort	27
2.9	Problema della selezione	28
3	Strutture dati	31
3.1	Pile	31
3.2	Code con priorità	32
3.3	Liste concatenate	33
3.4	Alberi binari	33
3.4.1	Alberi di ricerca	33
3.4.2	RB-alberi	33
3.4.3	B-alberi binomiali	33
3.5	Estensione di strutture dati	33
3.6	Heap binomiali	33
3.7	Insiemi disgiunti	33
3.8	Tabelle hash	33

4	Algoritmi per grafi	34
4.1	Grafi	34
4.2	Visita in ampiezza	38
4.3	Visita in profondità	42
4.4	Ordinamento topologico	46
4.5	Alberi di copertura di costo minimo	50
4.6	Cammini minimi a sorgente singola	54
4.7	Cammini minimi a sorgente multipla	58
4.8	Flusso massimo	58
5	Progettazione e analisi di algoritmi	59
5.1	Programmazione dinamica	59
5.2	Programmazione greedy	59
5.3	Programmazione lineare	59

Capitolo 1

Fondamenti

Il corso di Algoritmi ha l'obiettivo di fornire gli strumenti teorici e metodologici per la progettazione, descrizione e analisi formale di algoritmi. In particolare, studieremo come specificare in modo rigoroso una procedura di calcolo, come dimostrarne la correttezza e valutarne l'efficienza.

Definiamo **algoritmo** come una procedura di calcolo ben definita che prende uno o più valori in input per generarne ulteriori in output con un numero finito di passi. Di ogni algoritmo analizzeremo la **complessità**, intesa come funzione che misura le risorse necessarie alla sua esecuzione, intese tipicamente come tempo e spazio, in relazione alla dimensione dell'input. Questo ci permetterà di confrontare soluzioni alternative e scegliere quella più efficiente rispetto al problema considerato.

1.1 Complessità e notazione asintotica

La **complessità** di un algoritmo è una funzione che descrive la quantità di risorse necessarie alla sua esecuzione in relazione alla dimensione dell'input. Tipicamente la consideriamo in termini di **complessità temporale**, che riguarda il tempo di esecuzione, e di **complessità spaziale**, per la memoria utilizzata.

Non si tratta di un valore fisso, ma una funzione $T(n)$, dove n rappresenta la dimensione dell'input. Al variare di n , varia anche il numero di operazioni eseguite dall'algoritmo. Poiché siamo interessati al comportamento dell'algoritmo per input di grandi dimensioni, utilizziamo l'**analisi asintotica**, che permette di descrivere l'ordine di crescita della funzione trascurando costanti moltiplicative e termini di ordine inferiore. In particolare presenta le seguenti notazioni:

- **Limite asintotico superiore:** $O(f(n))$

Date due funzioni $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, diciamo che $f(n) \in O(g(n))$ se esistono una costante c e un valore $\bar{n}$ entrambi positivi tali che:

$$0 \leq f(n) \leq cg(n) \quad \forall n \geq \bar{n}$$

In tal caso g fornisce un limite superiore asintotico per f . Intuitivamente, per un n sufficientemente grande, la funzione f cresce al più come g , a meno di una costante moltiplicativa. Formalmente:

$$f(n) \in O(g(n)) \iff \exists c > 0, \exists \bar{n} : \forall n \geq \bar{n}, f(n) \leq cg(n)$$

• **Limite asintotico inferiore:** $\Omega(f(n))$

Date due funzioni $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, diciamo che $f(n) \in \Omega(g(n))$ se esistono una costante c e un valore $\bar{n}$ entrambi positivi tali che:

$$0 \leq cg(n) \leq f(n) \quad \forall n \geq \bar{n}$$

In questo caso g fornisce un limite inferiore asintotico per f . Formalmente:

$$f(n) \in \Omega(g(n)) \iff \exists c > 0, \exists \bar{n} : \forall n \geq \bar{n}, f(n) \geq cg(n)$$

• **Limite asintotico stretto:** $\Theta(f(n))$

Date due funzioni $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, diciamo che $f(n) \in \Theta(g(n))$ se per i valori costanti positivi $c_1, c_2, \bar{n}$ risulta:

$$0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \quad \forall n \geq \bar{n}$$

In questo caso g descrive l'ordine di crescita preciso di f , poiché il valore di quest'ultima sarà sempre compreso fra le altre due. Formalmente:

$$f(n) \in \Theta(g(n)) \iff f(n) \in O(g(n)) \wedge f(n) \in \Omega(g(n))$$

Esempio 1. Ricerca di un elemento in un array

La complessità della ricerca di un elemento in un array è data da una funzione lineare, consentendoci di modellarla come:

$$T(n) = cn + a$$

Dove c rappresenta il costo del confronto, n è la dimensione dell'input, mentre a rappresenta i costi iniziali, espressi in una costante. Lavorando con l'analisi asintotica possiamo rimuovere i termini costanti e considerare esclusivamente il valore variabile n , ottenendo:

- *Caso ottimo:* $T(n) \in \Theta(1)$, perché prende il primo elemento dell'array.
- *Caso pessimo:* $T(n) \in \Theta(n)$, perché scorre l'intero array fino all'elemento cercato.
- *Caso medio:* $T(n) \in \Theta(n)$, perché al netto di costanti, la complessità dipende dalla dimensione.

Esempio 2. Dimostrazione di appartenenza a $O(n)$

Sia la funzione $T(n) = 5n + 7 \in O(2n)$. Dobbiamo trovare un c tale per cui valga l'equazione

$$5n + 7 \leq c(2n)$$

Per far rispettare la relazione e quindi trovare un limite valido possiamo scegliere la costante $c = 3$, con la quale, svolgendo le operazioni, otteniamo:

$$5n + 7 \leq 6n$$

Risolvendo la disequazione infine otteniamo il valore $\bar{n} = 7$, dimostrando la correttezza della relazione.

Questi erano esempi concreti con funzioni ben definite, ma è possibile ragionare in termini più generali grazie alle seguenti proprietà:

• **Proprietà transitiva**

$$\begin{aligned} f(n) \in \Theta(g(n)) \wedge g(n) \in \Theta(h(n)) &\implies f(n) \in \Theta(h(n)) \\ f(n) \in O(g(n)) \wedge g(n) \in O(h(n)) &\implies f(n) \in O(h(n)) \\ f(n) \in \Omega(g(n)) \wedge g(n) \in \Omega(h(n)) &\implies f(n) \in \Omega(h(n)) \end{aligned} \quad (1.1)$$

• **Proprietà riflessiva**

$$\begin{aligned} f(n) &\in \Theta(f(n)) \\ f(n) &\in O(f(n)) \\ f(n) &\in \Omega(f(n)) \end{aligned} \quad (1.2)$$

• **Proprietà simmetrica**

$$f(n) \in \Theta(g(n)) \iff g(n) \in \Theta(f(n))$$

• **Simmetria trasposta**

$$f(n) \in O(g(n)) \iff g(n) \in \Omega(f(n))$$

Poiché queste proprietà valgono per le notazioni asintotiche, è possibile trarre un'analogia concettuale fra il confronto asintotico di funzioni e numeri reali. In particolare:

$$\begin{aligned} f(n) \in O(g(n)) &\equiv a \leq b \\ f(n) \in \Omega(g(n)) &\equiv a \geq b \\ f(n) \in \Theta(g(n)) &\equiv a = b \end{aligned} \quad (1.3)$$

Per la dimostrazione delle relazioni nei diversi ordini di grandezza ci serviremo del metodo di **sostituzione**, con lo scopo di ricavare la tesi tramite passaggi logici. Una buona prassi è data da:

1. Scrivere i dati in base alla loro definizione.
2. Identificare ciò che è necessario per la dimostrazione.
3. Verificare la tesi.

Esempio 3. Dimostrazione limite asintotico superiore

Supponiamo che la seguente formula sia vera. La si dimostri per confermare la supposizione:

$$f_1 \in O(g_1), f_2 \in O(g_2) \implies f_1 + f_2 \in O(g_1 + g_2)$$

Procediamo con i passaggi precedentemente menzionati:

1. *Riduciamo gli elementi alla loro definizione formale, ottenendo:*

$$f_1 \leq c_1 g_1(n); \quad f_2 \leq c_2 g_2(n)$$

2. *Per dimostrare la relazione è necessario trovare due valori c e $\bar{n}$ positivi. Ogni funzione ha i propri, quindi la c sarà data dalla loro somma, mentre $\bar{n}$ dal massimo di entrambi. Infatti:*

$$c = c_1 + c_2; \quad \bar{n} = \max(\bar{n}_1, \bar{n}_2)$$

3. *Procediamo con la verifica della tesi sostituendo agli elementi le relative definizioni:*

$$\begin{aligned} T(n) &= f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \\ &= f_1(n) + f_2(n) \leq c(g_1(n) + g_2(n)) \\ &= f_1(n) + f_2(n) \leq c(g_1 + g_2)(n) \implies f_1(n) + f_2(n) \in O(g_1 + g_2) \end{aligned} \quad (1.4)$$

Naturalmente per dare una visione completa della complessità di un algoritmo bisogna considerare anche il caso ottimo, quindi il suo limite asintotico inferiore. I passaggi di dimostrazione sono fondamentalmente gli stessi di prima.

Esempio 4. Dimostrazione limite asintotico inferiore

Dimostrare la seguente formula per confermare il limite inferiore dell'algoritmo:

$$f \in O(g(n)) \iff g(n) \in \Omega(f(n))$$

Stiamo asserendo che f è nell'ordine di grandezza di g se e solo se quest'ultima è il limite asintotico inferiore della prima. Procediamo:

1. *Definizione formale degli elementi:*

$$\exists c > 0, \exists \bar{n} : \forall n \geq \bar{n}, \quad f(n) \leq cg(n)$$

2. *Elementi necessari alla dimostrazione: I soliti valori $c, \bar{n}$.*

3. *Siccome la costante è positiva, effettuiamo il passaggio:*

$$cg(n) \geq f(n) \implies g(n) \geq \frac{f(n)}{c}$$

Infine diciamo che $\frac{1}{c} = c'$, che ci fa ottenere la definizione del limite inferiore:

$$g(n) \geq c'f(n) \implies g(n) \in \Omega(f(n))$$

Per riassumere, se vogliamo determinare l'ordine esatto di crescita è necessario dimostrare sia il **limite superiore** $O(f(n))$ che il **limite inferiore** $\Omega(f(n))$ rispetto alla stessa funzione, il quale è descritto da:

$$\theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

1.2 Equazioni di ricorrenza

Nella programmazione esistono due metodi principali per la costruzione di algoritmi: **ricorsione** e **iterazione**; in questa sezione studieremo entrambe le idee, con particolare attenzione agli algoritmi ricorsivi, rappresentati tramite le **equazioni di ricorrenza**. Una forma generale è data da:

$$T(n) = \begin{cases} \Theta(1) & n \leq \bar{n} \\ aT(n/b) + f(n) & n > \bar{n} \end{cases}$$

Dove, in particolare:

- **Numero dei sottoproblemi:** a .

- **Dimensione di ogni sottoproblema:** n/b .
- **Costo del lavoro svolto fuori dalle chiamate ricorsive:** $f(n)$.

Essendo che il caso base influisce esclusivamente su costanti additive e non sull'ordine asintotico, possiamo considerare solo il passo ricorsivo. Per risolvere queste equazioni ci serviremo del metodo di **sostituzione**, dove si suppone una stima asintotica della funzione, per poi provare a dimostrarla tramite induzione matematica, e degli **alberi di ricorrenza**, i quali la rappresentano con dei nodi la cui somma nel singolo livello è il costo della ricorsione in quel passo.

Esempio 5. Metodo di sostituzione

Supponiamo l'equazione di ricorrenza $T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n)$. Vogliamo dimostrare che il suo limite superiore è:

$$T(n) \in O(n \log n)$$

Assumiamo $T(n) \leq c(n \log n)$ per ogni $n \geq \bar{n}$ come ipotesi induttiva per trovare i vincoli da rispettare. Stabilire la verità di questa ipotesi è sufficiente a dimostrare la tesi.

Se il limite vale per $n \geq \bar{n}$, allora l'ipotesi varrà per $n = \lfloor n/2 \rfloor$, da cui, per definizione, possiamo dire:

$$T(n) \leq c(n \log n) \implies T(\lfloor n/2 \rfloor) \leq 2(c \cdot \lfloor n/2 \rfloor \log(\lfloor n/2 \rfloor)) + \Theta(n)$$

Da questa forma possiamo procedere con la risoluzione dell'equazione di ricorrenza:

$$\begin{aligned} T(n) &\leq 2(c \cdot \lfloor n/2 \rfloor \log(\lfloor n/2 \rfloor)) + \Theta(n) \\ &\leq c \cdot n \log(n/2) + \Theta(n) \\ &\leq c \cdot n \log(n) - c \cdot n \log(2) + \Theta(n) \\ &\leq c \cdot n \log(n) - cn + \Theta(n) \end{aligned} \tag{1.5}$$

Nell'ultimo passaggio abbiamo rimosso le costanti moltiplicative. Logicamente, questo è un valore minore dell'ipotesi, quindi possiamo dimostrare la veridicità della tesi affermando che:

$$c(n \log n) \geq cn \log(n) - cn + \Theta(n)$$

Esempio 6. Albero di ricorsione

Supponiamo l'equazione di ricorrenza $T(n) = 3T(n/4) + \Theta(n^2)$, definiamo il suo albero di ricorsione. Ad una prima occhiata è evidente che abbiamo 3 sottoproblemi e 4 come fattore di riduzione. Ragioniamo quindi per livelli:

- **Livello 0 - Radice**

$$\text{Nodi: } 1 \quad \text{Costo: } n^2$$

- **Livello 1**

Nodi: 3 Dimensione nodo: $n/4$ Costo nodo: $(n/4)^2$ Costo totale: $\frac{3}{16}n^2$

- **Livello i**

Nodi: 3^i Dimensione nodo: $n/4^i$ Costo nodo: $(n/4^i)^2$ Costo totale: $\left(\frac{3}{16}\right)^i n^2$

Infine determiniamo **altezza** dell'albero e **costo totale**. La prima è legata al totale di livelli della ricorsione. Siccome questa termina quando la somma dei nodi è uguale a 1, la ricaviamo con:

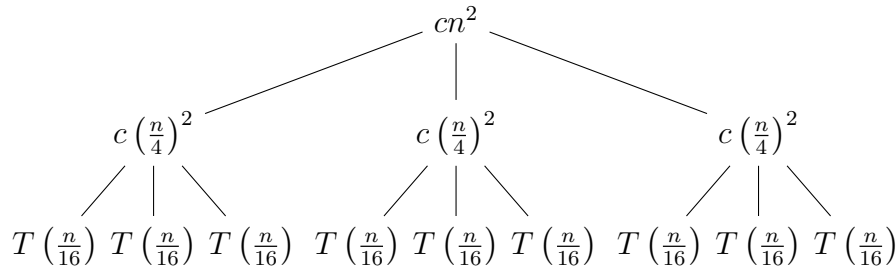
$$n/4^h = 1 \implies 4^h = n \implies h = \log_4(n)$$

Il costo totale si ottiene invece facendo la somma di tutti i nodi di ogni livello, quindi si descrive con una sommatoria che va da 0 al valore dell'altezza $\log_4(n)$, dunque:

$$\sum_{i=0}^{\log_4(n)} cn^2 \left(\frac{3}{16}\right)^i$$

Questa è allo stesso tempo una serie geometrica con ragione $3/16 < 1$. Questa converge, quindi il costo è dato dal primo termine:

$$T(n) \in \Theta(n^2)$$



Per quanto riguarda la **modalità iterativa**, usiamo la notazione $f^i(n)$ per denotare la funzione f applicata iterativamente i volte a un valore di entrata n . Formalmente, sia una funzione $f(n)$ definita sui reali; per ogni intero non negativo i definiamo ricorsivamente:

$$f^i(n) = \begin{cases} n & i = 0 \\ f(f^{i-1}(n)) & i > 0 \end{cases}$$

Semplicemente, $f(n) = 2n \implies f^i(n) = 2^i n$. Un esempio più complesso è il seguente.

Esempio 7. Metodo iterativo

Sia la funzione $T(n) = 1 + T(n-1)$. Per dimostrare $T(n) \in \Theta(n)$ è necessario esaurire tutte le iterazioni richieste.

$$\begin{aligned}
 T(n) &= 1 + T(n-1) \\
 &= 1 + 1 + T(n-2) \\
 &= 1 + 1 + 1 + T(n-3) \\
 &= 1 + \dots + 1 + T(n-i) \\
 &= 1 + \dots + 1 + T(n-n) \equiv 1 + \dots + 1 + T(1)
 \end{aligned} \tag{1.6}$$

Otteniamo dunque $T(n) = n \implies T(n) \in \Theta(n)$

1.3 Teorema dell'esperto

Il **Teorema dell'esperto**, master theorem o metodo principale, è un enunciato utile per imporre una dominazione stretta su una funzione $f(n)$ tramite un polinomio di mezzo, definito come n^ϵ . Pone una condizione sufficiente ma non necessaria per le dimostrazioni e consente di risolvere più facilmente le ricorrenze in forma:

$$T(n) = aT(n/b) + f(n)$$

Poniamo una **funzione spartiacque** $n^{\log_b a}$, usata per effettuare un confronto. Se una funzione è dominata da un'altra che sta sotto alla spartiacque, avremo la garanzia che la prima si troverà nell'ordine di grandezza di quest'ultima. Il teorema si definisce formalmente come:

Teorema 1. Teorema dell'esperto

Siano $a > 0, b > 1$ delle costanti e $f(n)$ una funzione forzante definita e non negativa per tutti i reali maggiori di una certa soglia. Sia inoltre $T(n)$ la ricorrenza definita per $n \in \mathbb{N}$ da:

$$T(n) = aT(n/b) + f(n)$$

Dove l'espressione $aT(n/b) = a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ per opportune costanti $a', a'' \geq 0 : a = a' + a''$. Allora il comportamento asintotico della ricorrenza può essere caratterizzato nel seguente modo:

1. Se esiste una costante $\epsilon > 0 : f(n) \in O(n^{\log_b(a-\epsilon)})$, allora $T(n) \in \Theta(n^{\log_b a})$.
2. Se esiste una costante $k \geq 0 : f(n) \in \Theta(n^{\log_b a} \cdot \log^k n)$, allora $T(n) \in \Theta(n^{\log_b a} \cdot \log^{k+1} n)$.

3. Se esiste una costante $\epsilon > 0$: $f(n) \in \Omega(n^{\log_b(a+\epsilon)})$ e inoltre $f(n)$ soddisfa la seguente **condizione di regolarità**:

$$af(n/b) \leq cf(n) \forall n \text{ abbastanza grandi e una } c < 1$$

Allora $T(n) \in \Theta(f(n))$.

Il teorema, tuttavia, non si può usare in ogni istanza. I casi da evitare sono:

- La ricorrenza non è della forma $aT(n/b) + f(n)$.
- C'è una divisione irregolare, come $T(n-1)$.
- $f(n)$ non è confrontabile con un polinomio.
- Manca la condizione di regolarità nel caso 3.

Esempio 8. Caso 1 del teorema

Sia $T(n) = 9T(n/3) + n$, abbiamo $a = 9, b = 3 \implies n^{\log_3 9} = n^2$. Abbiamo inoltre $f(n) = n$; confrontandola con l' risulta:

$$n \in O(n^{2-\epsilon})$$

Ci troviamo quindi nel primo caso e confermiamo che $T(n) \in \Theta(n^2)$.

Esempio 9. Caso 2 del teorema

Sia $T(n) = T(2/3n) + 1$, abbiamo $a = 1, b = 3/2, f(n) = 1$, quindi:

$$n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$$

Siccome $f(n) = 1 \implies f(n) \in \Theta(1)$ ci troviamo nel caso 2. Ogni qualvolta si riporta il problema a costanti, ci troviamo in ordine di $\log(n)$, dato che rappresenta il numero dei livelli dell'albero. Confermiamo che $T(n) \in \Theta(\log(n))$.

Esempio 10. Caso 3 del teorema

Sia $T(n) = 3T(n/4) + n \log(n)$, abbiamo $a = 3, b = 4, f(n) = n \log(n)$, quindi:

$$n^{\log_b a} = n^{\log_4 3}$$

Notiamo che $n \log(n)$ cresce più rapidamente della spartiacque $n^{\log_4 3}$, dunque ci troviamo nel caso 3. Verifichiamo prima la condizione di regolarità, prendendo per esempio $c = 3/4$:

$$3(n/4) \log(n/4) \leq c \cdot n \log(n)$$

La condizione è vera per n grande con $c < 1$. Dunque $T(n) = \Theta(n \log(n))$

Esempio 11. Teorema non applicabile

Sia $T(n) = 2T(n/2) + n \log(n)$, abbiamo $a = 2, b = 2, f(n) = n \log(n)$, quindi:

$$n^{\log_b a} = n^1 = n$$

Notiamo che non ci troviamo in nessuno dei tre casi precedenti, perché la funzione $f(n)$ cresce più rapidamente di qualunque potenza positiva, quindi il teorema dell'esperto non è applicabile.

1.4 Esercizi svolti

Fondamentalmente servono:

1. Dimostrazione limiti asintotici superiori ed inferiori
2. Equazioni di ricorrenza con sostituzione, iterazione, alberi di ricorrenza e teorema dell'esperto

Capitolo 2

Ordinamento e selezione

2.1 Problema dell'ordinamento

Il problema dell'ordinamento dei dati in una struttura è uno dei più comuni che si incontra nella pratica e consente di introdurre vari strumenti di analisi e tecniche di progettazione standard. Formalmente viene definito come:

- **Input:** Sequenza di n oggetti $\langle a_1, \dots, a_n \rangle$ sulla quale è definita una relazione di ordine totale.
- **Output:** Permutazione $\langle a'_1, \dots, a'_n \rangle$ della sequenza di input tale che $a'_1 \leq \dots \leq a'_n$.

In pratica, data per esempio una sequenza di numeri in input $\langle 1, 4, 2, 7, 5, 3 \rangle$, l'algoritmo risolutivo ritornerà in output $\langle 1, 2, 3, 4, 5, 7 \rangle$. La sequenza specifica in entrata è detta **istanza** del problema e per essere elaborata è necessario che appartenga al dominio per cui l'algoritmo è definito.

Diciamo che un algoritmo è **corretto** se, per ogni istanza valida del problema, termina e produce un output conforme alla specifica e sebbene si tratti di un concetto intuitivamente semplice, esistono molti algoritmi adatti a questo compito, la cui scelta è determinata da alcuni fattori come la **dimensione** del numero di dati, il loro **ordinamento iniziale**, eventuali **vincoli** o anche limiti fisici dell'elaboratore.

2.2 Insertion Sort

Algoritmo efficiente per ordinare un piccolo numero di elementi, opera allo stesso modo in cui si ordinerebbe un mazzo di carte. Dato in input un array A , mantengo una parte ordinata ed una non ordinata; ad ogni iterazione si prende un elemento dalla seconda e lo si inserisce nella posizione corretta, trovata confrontando tutte le carte a partire dall'inizio.

- **Caso pessimo:** $\Theta(n^2)$
- **Caso ottimo:** $\Theta(n)$. Si ottiene quando l'array è già ordinato.
- **Caso medio:** $\Theta(n^2)$
- **Ordinamento in loco:** Sì
- **Stabilità:** Sì

Algorithm 1 InsertionSort(A,n)

Require: A: Array, n: Dimensione array

```

1: for  $i \leftarrow 2$  to  $n$  do                                ▷ Scorre da startPos+1=2 fino a n dimensione
2:    $key \leftarrow A[i]$ 
3:    $j \leftarrow i - 1$                                        ▷ L'indice j è usato per la sezione ordinata. Valori: [1,n]
4:
5:   while  $j > 0$  AND  $A[j] > key$  do                        ▷ Non sei all'inizio e key minore di A[i]
6:      $A[j + 1] \leftarrow A[j]$  ▷ Sposta a destra di una posizione il valore maggiore di key
7:      $j \leftarrow j - 1$                                      ▷ Spostati a sinistra di una posizione
8:   end while
9:    $A[j + 1] \leftarrow key$                                   ▷ Inserisci key nella posizione corretta
10: end for

```

La valutazione della complessità di questo algoritmo vede analizzati due elementi principali: il ciclo **for**, avente tre istruzioni (gli assegnamenti) di complessità costante, e il ciclo **while** interno al primo, il quale, potendo scorrere potenzialmente l'intero array ad ogni iterazione, aumenta notevolmente i tempi di esecuzione.

Dove richiede un tempo di lavoro generalmente alto rispetto ad altri algoritmi, la sua potenzialità sta nel risparmio della memoria; infatti richiede solo tre variabili per essere eseguito e non richiede alcun supporto esterno per la gestione dell'array. Diciamo, per questa ragione, che l'algoritmo **opera in loco**. Un'altra sua caratteristica è che gli elementi uguali non vengono riordinati fra di loro poiché il confronto effettuato è stretto. In queste condizioni, diciamo che l'algoritmo è **stabile**.

2.3 Merge Sort

Algoritmo basato sulla tecnica del **Divide et Impera**, ovvero nella suddivisione del problema in sottoproblemi della stessa natura di dimensione via via più piccola.

Diviso l'array di partenza in due parti uguali, si ordinano entrambe applicando ricorsivamente l'algoritmo, per poi combinare queste due nuove sezioni con un sottoalgoritmo

merge di supporto, prendendo volta per volta il valore più piccolo contenuto in entrambi, fino ad esaurire tutti i valori.

- **Caso pessimo:** $\Theta(n \log(n))$
- **Caso ottimo:** $\Theta(n \log(n))$
- **Caso medio:** $\Theta(n \log(n))$
- **Ordinamento in loco:** No, richiede memoria ausiliaria.
- **Stabilità:** Sì

Algorithm 2 merge(A , left, center, right)

Require: A : Array, left: Posizione iniziale, center: Centro, right: Posizione finale

```

1:  $i \leftarrow \text{left}$ 
2:  $j \leftarrow \text{center} + 1$ 
3:  $k = 1$  ▷ Indice dell'array di supporto
4:  $B[\text{right} - \text{left} + 1]$  ▷ Array di supporto di dimensione right-left+1
5:
6: while  $i \leq \text{center}$  AND  $j \leq \text{right}$  do ▷ Fintanto che i sottarray non sono finiti
7:   if  $A[i] \leq A[j]$  then ▷ Se l'elemento del primo è minore, copia quello e avanza
8:      $B[k] \leftarrow A[i]$ 
9:      $i \leftarrow i + 1$ 
10:     $k \leftarrow k + 1$ 
11:   else ▷ Altrimenti procedimento analogo, ma per il secondo array
12:      $B[k] \leftarrow A[j]$ 
13:      $j \leftarrow j + 1$ 
14:      $k \leftarrow k + 1$ 
15:   end if
16: end while
17: while  $i \leq \text{center}$  do ▷ Copia il primo array ordinato
18:    $B[k] \leftarrow A[i]$ 
19:    $i \leftarrow i + 1$ 
20:    $k \leftarrow k + 1$ 
21: end while
22: while  $j \leq \text{right}$  do ▷ Copia il secondo array ordinato
23:    $B[k] \leftarrow A[j]$ 
24:    $j \leftarrow j + 1$ 
25:    $k \leftarrow k + 1$ 
26: end while
27: for  $k \leftarrow \text{left}$  to right do ▷ Sovrascrivi l'array di base con quello ordinato
28:    $a[k] \leftarrow b[k - \text{left}]$ 
29: end for

```

Algorithm 3 mergeSort(A, left, right)**Require:** A: Array, left: Posizione iniziale, right: Posizione finale

```

1: if left < right then
2:   center = (left+right)/2
3:   mergeSort(A, left, center)           ▷ Ordina la parte prima della metà
4:   mergeSort(A, center+1, right)       ▷ Ordina la parte dopo la metà
5:   merge(A, left, center, right)       ▷ Fondi i due sottoarray ordinati
6: end if

```

Diversamente rispetto ad Insertion Sort, per valutare la complessità di questo algoritmo bisogna considerare la ricorsione. Sapendo che ad ogni passo ricorsivo il problema è diviso in **due sottoproblemi** ($n/2$), dove si effettuano **due chiamate ricorsive** ($2T$), e che il merge scorre tutti gli elementi **almeno una volta** $\Theta(n)$, possiamo dire che Merge Sort è rappresentato da:

$$T(n) = 2T(n/2) + \Theta(n)$$

Possiamo usare il teorema dell'esperto per la dimostrazione. Convertendo la ricorrenza alla forma generale, otteniamo:

$$a = 2, \quad b = 2, \quad f(n) = \Theta(n), \quad \log_2(2) = 1 \implies n^1 = n$$

Confrontando $f(n)$ con la spartiacque, notiamo che appartengono allo stesso ordine di grandezza, facendoci trovare nel caso 2 del teorema, infatti:

$$f(n) = \Theta(n^{\log_b(a)}) \implies T(n) = \Theta(n \log(n))$$

2.4 Struttura Heap e Heap Sort

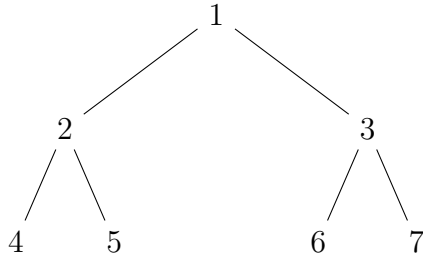
Introduciamo ora la prima struttura di dati del corso. Nelle sezioni precedenti è stato menzionato il concetto di **albero binario**, una struttura composta in cima da un nodo, detto **radice**, il quale, in quanto binario, si dirama in due direzioni verso il basso attraverso **archi**, creando due nodi figli. In particolare, quando un nodo figlio è alla base dell'albero si dice **foglia**.

Come è possibile intuire, gli alberi binari sono organizzati in livelli, i quali indicano la **profondità**, ovvero la distanza dalla radice. Quando un livello ha il numero massimo di nodi ivi ottenibili si dice **completo**. Tuttavia possono esistere alberi **semi-completi**, quando la base non è completa.

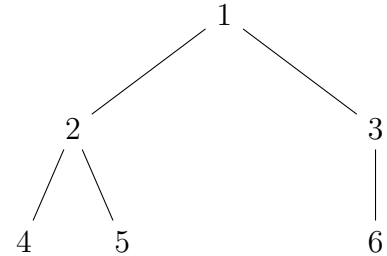
Un vantaggio di questa struttura è la semplicità della ricerca dei nodi; infatti, la dinamica si dispiega in tre casi:

- **Per figlio sinistro:** $\text{indice} \cdot 2 + 0$

- **Per figlio destro:** $\text{indice} \cdot 2 + 1$
- **Da figlio a padre:** $\text{indice}/2$



Albero binario completo



Albero binario semi-completo

Un **heap** è una struttura dati implementata tramite un array, considerata sotto forma di un albero binario semi-completo, i cui nodi rappresentano gli elementi al suo interno. Deve inoltre rispettare la proprietà che lo contraddistingue: in un **MAX-Heap** ogni nodo è maggiore o uguale ai propri figli. Analogamente, è possibile definire un **MIN-Heap** dove vale il contrario. Nella gestione di questa struttura dati, tuttavia, in necessità di inserimento nodi, non necessariamente si potrà rispettare sempre la proprietà, ragion per cui utilizziamo un algoritmo **heapify()** ad-hoc, con lo scopo di prendere questo problema locale e trasferirlo alle foglie, dove sarà risolto.

Algorithm 4 heapify(A, i)

Require: A: Array, i: Indice

```

1:  $l \leftarrow \text{left}(i)$ ,  $r \leftarrow \text{right}(i)$            ▷ l prende figlio-sx, r prende figlio-dx
2:
3: if  $l \leq A.\text{heapSize}$  AND  $A[l] > A[i]$  then     ▷ Se figlio-sx ha valore > i, non è heap.
4:    $\text{largest} \leftarrow l$ 
5: else
6:    $\text{largest} \leftarrow i$                              ▷ Altrimenti segna che il più grande è i, come inteso.
7: end if
8:
9: if  $r \leq A.\text{heapSize}$  AND  $A[r] > A[\text{largest}]$  then   ▷ Controllo figlio destro
10:   $\text{largest} \leftarrow r$ 
11: end if
12:
13: if  $\text{largest} \neq i$  then                               ▷ Se la proprietà non è valida
14:    $\text{switch}(A[i], A[\text{largest}])$  ▷ Scambia la posizione dei due nodi, largest è l'anomalia
15:    $\text{heapify}(A, \text{largest})$  ▷ Sposta ricorsivamente largest finché non rispetta la regola
16: end if
  
```

La complessità di questo algoritmo è equivalente alla sua profondità, quindi il caso peggiore scende lungo l'altezza dell'albero fino alle foglie, dando come tempo di esecuzione $T(n) \in O(\log(n))$.

Abbiamo detto che gli heap sono basati sugli array, ma è necessario che questi siano trasformati in alberi binari che rispettano la regola, per questo introduciamo l'algoritmo **buildHeap()**, il quale trasforma un array di dimensione n in un heap chiamando la procedura **heapify()** in stile bottom-up. Sapendo che tutti gli elementi del sottoarray $A[(n/2) + 1 : n]$, ovvero della seconda metà, sono foglie dell'albero perché per ogni $i > n/2$ vale $2i > n$, possono essere considerate come radici di un heap con un solo elemento. **Heapify()** attraverserà i nodi restanti inserendoli opportunamente.

Algorithm 5 buildHeap(A, n)

Require: A : Array, n : Dimensione array

$A.\text{heapSize} \leftarrow n$

for $i \leftarrow n/2$ **down to** 1 **do** ▷ La seconda metà dell'array rispetta già la proprietà
 heapify(A,i)

end for

Determinare una stima del limite superiore di questo algoritmo è semplice: sappiamo che si effettuano $O(n)$ chiamate alla procedura **heapify()**, la quale ha tempo di esecuzione $O(\log(n))$. Dunque il tempo di esecuzione di **buildHeap()** è $O(n\log(n))$.

Se necessario, è possibile farne un'analisi più rigorosa osservando che il tempo per eseguire **heapify()** su un nodo varia in base all'altezza dello stesso nell'albero, quindi $O(h)$. In questi termini, aggiungendo la costante c per la notazione asintotica, possiamo dire che il costo totale di **buildHeap()** è limitato superiormente da:

$$\sum_{h=0}^{\log(n)} \lceil n/2^{h+1} \rceil ch$$

Eseguiamo ora i seguenti passaggi, fattibili grazie al funzionamento delle serie geometriche convergenti:

$$\begin{aligned} \sum_{h=0}^{\lfloor \log(n) \rfloor} \left\lfloor \frac{n}{2^{h+1}} \right\rfloor ch &\leq \sum_{h=0}^{\lfloor \log(n) \rfloor} \frac{n}{2^h} ch = cn \sum_{h=0}^{\lfloor \log(n) \rfloor} \frac{h}{2^h} \\ &\leq cn \sum_{h=0}^{\infty} \frac{h}{2^h} \\ &\leq cn \cdot \frac{1/2}{(1 - 1/2)^2} = O(n) \end{aligned} \tag{2.1}$$

Dunque è possibile costruire un heap a partire da un array non ordinato in **tempo lineare**. Ciò vale sia per MAX-heap che MIN-heap.

Quanto appena detto è utilizzabile per risolvere il problema dell'ordinamento tramite l'algoritmo **heapSort()**. Inizia chiamando **buildHeap()** per trasformare un array di dimensione n in una struttura heap. In quanto l'elemento più grande dell'array sta in $A[1]$, lo si può inserire nella posizione corretta scambiandolo con $A[n]$. Tuttavia, riducendo la dimensione dell'albero, "scartando" il nodo n , i sottoalberi radicati nei figli potrebbero non rispettare la proprietà, quindi si chiama **heapify()**, che termina lasciando nell'array $A[1 : n - 1]$ un heap.

Questo procedimento è ripetuto per l'heap di dimensione $n - 1$ fino ad ottenere un heap di grandezza 2.

- **Caso pessimo:** $\Theta(n \log(n))$
- **Caso ottimo:** $\Theta(n \log(n))$
- **Caso medio:** $\Theta(n \log(n))$
- **Ordinamento in loco:** Sì
- **Stabilità:** No. Dato un array con nodi di valore uguale, li scambia.

Algorithm 6 heapSort(A,n)

Require: A : Array, n : Dimensione array
 buildHeap(A,n)

for $i \leftarrow n$ down to 2 do switch($A[1]$, $A[i]$) $A.\text{heapSize} \leftarrow A.\text{heapSize} - 1$ heapify (A,1) end for	▷ Decrementa n fino a dimensione 2 ▷ Scambia la posizione di $A[1]$ con $A[i]$ ▷ Scartato il nodo, riduci la dimensione ▷ Assicurati del rispetto della proprietà
---	--

Per valutare la complessità bisogna ragionare sulle procedure chiamate. **BuildHeap** impiega un tempo lineare $O(n)$, lo **switch** ha tempo costante $O(1)$ e ciascuna delle $n - 1$ chiamate a **heapify()** usa un tempo $O(\log(n))$. Quindi **heapSort()** impiega un tempo $T(n) = O(n \log(n))$.

2.5 Quick Sort classico e probabilistico

Come per **mergeSort()**, **quickSort()** è un algoritmo di ordinamento basato sul metodo del Divide et Impera, si tratta della soluzione più comune ed efficiente per la risoluzione

degli ordinamenti. Intuitivamente, dato un array con una certa dimensione, lo dividiamo in due parti in base ad un dato valore perno, ottenendo la **sezione bassa**, dove ogni elemento è minore o uguale ad esso, e la **sezione alta**, con tutti i valori maggiori o uguali al perno. Si procede poi con un'ordinamento delle due parti chiamando ricorsivamente la procedura, le quali non necessitano di essere ricombinate grazie alla logica usata per dividere l'array iniziale.

- **Caso pessimo:** $\Theta(n^2)$, accade quando il pivot scelto è il valore minimo o massimo dell'array, causando partizionamenti sbilanciati, dove un sub-array è vuoto e l'altro ha tutti gli elementi.
- **Caso ottimo:** $\Theta(n \log(n))$
- **Caso medio:** $\Theta(n \log(n))$
- **Ordinamento in loco:** Sì
- **Stabilità:** No, la funzione `switch()` scambia gli elementi uguali.

Algorithm 7 quickSort(A, startPos, endPos)

Require: A: Array, startPos: Posizione iniziale, endPos: Posizione finale

```

if startPos < endPos then
    q = partition(A, startPos, endPos)  ▷ Partiziona l'array e ritorna l'indice pivot
    quickSort(A, startPos, q-1)         ▷ Ordina la sezione bassa
    quickSort(A, q+1, endPos)           ▷ Ordina la sezione alta
end if

```

Algorithm 8 partition(A, startPos, endPos)

Require: A: Array, startPos: Posizione iniziale, endPos: Posizione finale

```

x ← A[endPos]                                     ▷ Il pivot
i ← startPos - 1                                  ▷ L'indice più alto della sezione bassa

for j ← startPos to endPos - 1 do                ▷ Gestisce ogni elemento non pivot
    if A[j] ≤ x then                                ▷ Controlla se l'elemento è in sezione bassa
        i ← i + 1                                    ▷ In tal caso incrementa la posizione...
        switch(A[i], A[j])                          ▷ E inserisce l'elemento nella nuova posizione
    end if
end for

switch(A[i+1], A[endPos])                         ▷ Il pivot si trova ora all'inizio della sezione alta
return i+1                                         ▷ Ritorna il nuovo indice del pivot

```

Per analizzare il tempo di esecuzione di questo algoritmo è bene osservare i casi possibili. Per esempio, il caso pessimo si ottiene quando l'elemento scelto come pivot è un massimo o un minimo dell'insieme, facendo risultare una sezione completamente vuota e l'altra con un sub-array uguale all'originale. In questi termini la ricorrenza si indica con:

$$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n)$$

Usando il metodo di iterazione possiamo provare che il tempo di esecuzione in queste condizioni è quadratico, considerando $\Theta(n) = n$:

$$\begin{aligned} T(n) &= \Theta(n) + T(n-1) \\ &= n + (n-1) + T(n-2) \\ &= n + (n-1) + (n-2) + T(n-3) \\ &= \sum_{i=1}^n i = \frac{n(n+1)}{2} \implies T(n) \in \Theta(n^2) \end{aligned} \tag{2.2}$$

Avendo un caso pessimo estremamente lento, il trucco per eludere questa istanza si è trovato nella casualità. Se infatti si effettua una partizione su un array il cui pivot è scelto casualmente, è remota la probabilità di incappare nel caso pessimo. Introduciamo dunque **randPartition()**.

Algorithm 9 randPartition(A, startPos, endPos)

$i \leftarrow \text{random}(\text{startPos}, \text{endPos})$ ▷ Sceglie un pivot casuale
 switch(A[endPos], A[i])
return partition(A, startPos, endPos)

La definizione di quickSort() con questo tipo di partizione non cambia la sua logica, ma naturalmente sarà necessario ridefinire i metodi per la divisione dei sub-array. Per quanto riguarda lo studio della complessità, invece, è necessario ragionare in termini probabilistici, poiché stiamo lavorando con la casualità.

Avendo la probabilità $1/n$ di pescare un valore perno, possiamo definire la ricorrenza di randomized quickSort() come l'espressione del valore atteso di una variabile aleatoria:

$$T(n) = \mathbb{E}[T(n)] = \Theta(n) + \frac{1}{n} \sum_{i=1}^n (T(i-1) + T(n-i))$$

Dove $\Theta(n)$ è il costo della partizione, $i-1$ la dimensione della sezione bassa, mentre $n-i$ è lo stesso della sezione alta. Notiamo inoltre una particolarità: nella sommatoria, ad ogni passo $T(i-1)$ cresce, mentre $T(n-i)$ decresce. Dunque, in questa istanza, il termine

$T(k)$ compare due volte ogni iterazione, consentendo di rappresentare la ricorrenza del valore atteso come:

$$2 \sum_{k=0}^{n-1} T(k) \implies T(n) = \Theta(n) + \frac{2}{n} \sum_{i=1}^{n-1} T(i)$$

2.6 Algoritmi di ordinamento lineari e Counting Sort

Finora sono stati trattati algoritmi di **ordinamento per confronti**, ovvero che operano secondo una logica esplicabile tramite un albero decisionale, ovvero un albero binario completo, dove i nodi interni sono visti come confronti e le foglie, in totale $n!$, come determinate permutazioni dell'input. Questo perché l'albero di altezza h ha al massimo 2^h foglie, dunque vale $2^h \geq n!$.

Essendo tuttavia l'altezza logaritmica, possiamo porre la seguente disuguaglianza, dimostrando il limite inferiore per questo tipo di algoritmo grazie alle proprietà dei logaritmi:

$$\begin{aligned} h &\geq \log(n!) \\ &\geq \log(n^n) \\ &\geq n \log(n) \end{aligned} \tag{2.3}$$

Quindi `mergeSort()` e `heapSort()` sono algoritmi di ordinamento per confronti asintoticamente ottimali. In queste condizioni. Perché è possibile raggiungere tempi ulteriormente inferiori aggiungendo delle informazioni di base prima di attuare la procedura.

Per esempio, supponiamo di avere un array di numeri interi la cui lunghezza k è arbitraria e conosciuta; possiamo contare gli elementi per ricavarne la loro posizione corretta. Questa è la logica dietro a **countingSort()**.

- **Caso pessimo:** $\Theta(\max(n, k))$
- **Caso ottimo:** $\Theta(\max(n, k))$
- **Caso medio:** $\Theta(\max(n, k))$
- **Ordinamento in loco:** No, ritorna un array supplementare
- **Stabilità:** Sì

Algorithm 10 countingSort(A, n, k)**Require:** A : Array, n : Dimensione A , k valore numero massimo

```

for  $i \leftarrow 0$  to  $k$  do
     $C[i] \leftarrow 0$                                 ▷ Inizializza array  $C$ 
end for

for  $j \leftarrow 1$  to  $n$  do                        ▷ Così  $C[i]$  ha numero di elementi uguali ad  $i$ 
     $C[A[j]] \leftarrow C[A[j]] + 1$ 
end for
for  $i \leftarrow 1$  to  $k$  do                        ▷ Così  $C[i]$  ha numero di elementi minori o uguali ad  $i$ 
     $C[i] \leftarrow C[i] + C[i - 1]$ 
end for

for  $j \leftarrow n$  down to  $1$  do                ▷ Copia  $A$  in  $B$  partendo dal fondo di  $A$ 
     $B[C[A[j]]] \leftarrow A[j]$ 
     $C[A[j]] \leftarrow C[A[j]] - 1$     ▷ Garanzia di stabilità, inserisce il numero in currPos-1
end for
return  $B$ 

```

Non è il più amichevole degli algoritmi sintatticamente parlando, quindi aggiungo anche un esempio intuitivo. Supponiamo di avere un array A da ordinare e un array B , ritornato dalla procedura. Naturalmente, avremo che

$$A = \{3, 6, 4, 1, 3, 4, 1, 4\}, \quad B = \{1, 1, 3, 3, 4, 4, 4, 6\}$$

Come prima operazione si inizializza l'array di supporto C , il quale avrà dimensione uguale al valore massimo presente in A :

$$C = \{0, 0, 0, 0, 0, 0\}$$

Dopodiché, si conteranno le occorrenze dei valori di A , incrementando il relativo totale di occorrenze salvato nell'indice corrispondente al loro valore. Per esempio, se si legge il valore $n = 1$, incrementeremo il totale di $C[1]$, o meglio $C[n]$, ottenendo, a fine del secondo for:

$$C = \{2, 0, 2, 3, 0, 1\}$$

Al terzo ciclo for calcoliamo quanti elementi sono minori o uguali dell'indice corrente, sommando il valore dell'indice attuale con quello precedente. Otteniamo, dunque:

$$\begin{aligned}
 C[2] &= C[2] + C[1] = 2 + 0 \implies C = \{2, 2, 2, 3, 0, 1\} \\
 C[3] &= C[3] + C[2] = 2 + 2 \implies C = \{2, 2, 4, 3, 0, 1\} \\
 C[4] &= C[4] + C[3] = 3 + 4 \implies C = \{2, 2, 4, 7, 0, 1\} \\
 C[5] &= C[5] + C[4] = 0 + 7 \implies C = \{2, 2, 4, 7, 7, 1\} \\
 C[6] &= C[6] + C[5] = 1 + 7 \implies C = \{2, 2, 4, 7, 7, 8\}
 \end{aligned} \tag{2.4}$$

Infine si andrà a copiare l'array A in B , sfruttando gli indici per C dati dai valori di A , ottenendo, finalmente:

$$B = \{1, 1, 3, 3, 4, 4, 4, 6\}$$

Un dettaglio sottile a cui fare attenzione è che, essendo la dimensione di C dipendente dal valore massimo contenuto in A , sarebbe possibile incappare in un'istanza con un valore particolarmente grande. In questo caso, essendo l'algoritmo in ordine di grandezza lineare, eseguirà tante operazioni quante questo valore, rendendolo estremamente inefficiente e quindi impraticabile.

2.7 Radix Sort

RadixSort() è un algoritmo di ordinamento usato spesso per gli ordinamenti di record con più campi chiave, come per esempio delle date. Intuitivamente, dato un array di numeri interi, ordina iterativamente l'intero vettore a partire dalla cifra meno significativa; ad ogni elaborazione dell'intero array si passa alla cifra precedente finché l'algoritmo non è ultimato.

- **Caso pessimo:** $\Theta(d(n + k))$, con d totale delle cifre e k la base del numero.
- **Caso ottimo:** $\Theta(d(n + k))$
- **Caso medio:** $\Theta(d(n + k))$
- **Ordinamento in loco:** Dipende dall'ordinamento usato.
- **Stabilità:** Sì, se l'algoritmo interno è stabile.

Algorithm 11 radixSort(A, n, d)

Require: A : Array, n : Lunghezza array, d : Numero di cifre degli elementi
for $i = 1$ **to** d **do**
 Qualunque ordinamento stabile per ordinare $A[1 : n]$
end for

Dove è possibile scegliere l'algoritmo di ordinamento all'interno del ciclo for, generalmente è utilizzato countingSort(), poiché consente di rendere il processo più efficiente rispetto ad altri. Per visualizzarne il funzionamento, segue esempio:

1. **Stato iniziale dell'array:** [329, 457, 657, 839, 436, 720, 355]
2. **Risultato della prima iterazione:** [720, 355, 436, 457, 657, 329, 839]
3. **Risultato della seconda iterazione:** [720, 329, 436, 839, 355, 457, 657]
4. **Risultato dell'ultima iterazione:** [329, 355, 436, 457, 657, 720, 839]

2.8 Bucket Sort

Il **BucketSort()** è un algoritmo che presuppone un input generato da un processo casuale, distribuito uniformemente e indipendentemente nell'intervallo $[0, 1)$. Intuitivamente divide l'intervallo in un totale di n "secchi" equiprobabili, per poi associare i valori al rispettivo bucket. Dopodiché, si ordinano questi ultimi con un tempo lineare, per poi concatenarli e ottenere il risultato. È verosimile aspettarsi in media un elemento per secchio. In questi termini, è necessario ragionare con le variabili aleatorie di Bernoulli, definite come:

$$X_{i,j} = \begin{cases} 1 & A[i] \in \text{Bucket}_j \\ 0 & \end{cases} \quad ; \quad \mathbb{E}[X_j] = 1$$

Indichiamo dunque il valore positivo quando l'elemento si trova nel rispettivo secchio. Insieme a questo, definiamo il numero di elementi nel bucket a indice j , ovvero quelli dove la variabile aleatoria vale 1 come:

$$X_j = \sum_i X_{i,j}; \quad P(X_{i,j} = 1) = \frac{1}{n}$$

Questa sommatoria determina la complessità, mentre per determinare il caso medio si calcola il suo valore atteso. I bucket vuoti non sono sommati.

- **Caso pessimo:** $\Theta(n^2)$
- **Caso ottimo:** $\Theta(n)$
- **Caso medio:** $\Theta(n)$
- **Ordinamento in loco:** No, usa un array di supporto.
- **Stabilità:** Dipende dall'algoritmo usato per ordinare i bucket.

Algorithm 12 bucketSort(A, n)

Require: A : Array, n : Dimensione array

$B[0 : n - 1]$

▷ Nuovo array vuoto con dim: $[0:n-1]$

for $i \leftarrow 1$ **to** n **do** Inserisci $A[i]$ nell'array $B[[n \cdot A[i]]]$

end for

for $i \leftarrow 1$ **to** n **do** Ordina l'array B con un algoritmo di ordinamento

end for

Concatena i bucket

return B

Per l'analisi della complessità di questo algoritmo bisogna considerare che tutte le operazioni effettuate, eccezion fatta per l'ordinamento dei bucket, hanno costo $O(n)$ nel caso peggiore. Il tempo totale delle n chiamate eseguite dall'algoritmo di ordinamento, tipicamente `insertionSort()`, è $O(n^2)$. Se indichiamo X_j la variabile aleatoria dei bucket inseriti nell'array B , possiamo affermare che il tempo di esecuzione di `bucketSort()` è dato da:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(X_j^2)$$

Come detto prima, per dimostrare il tempo di esecuzione medio, è necessario considerare il valore atteso della sommatoria dei bucket. Questo grazie alla proprietà di linearità del valore atteso, infatti:

$$\begin{aligned} \mathbb{E}[T(n)] &= \mathbb{E} \left[\Theta(n) + \sum_{j=0}^{n-1} O(X_j^2) \right] \\ &= \Theta(n) + \sum_{j=0}^{n-1} \mathbb{E}[O(X_j^2)] \\ &= \Theta(n) + \sum_{j=0}^{n-1} O(\mathbb{E}[X_j^2]) \end{aligned} \tag{2.5}$$

Dove, sapendo che

$$\mathbb{E}[X_j] = \sum_i E[X_{ij}] = n(1/n) = 1; \quad \text{Var}[X_j] = \sum_i \text{Var}[X_{ij}] = n(1/n)(1-1/n) = 1-1/n$$

Possiamo affermare che:

$$E[X_j^2] = \text{Var}[X_j] + E^2[X_j] = 1-1/n + 1^2 = 2-1/n; \quad \sum_j E[X_j^2] \implies n(2-1/n) = 2n-1$$

Dimostrando la sommatoria inserita nell'equazione iniziale, pertanto, sotto l'ipotesi di distribuzione uniforme e indipendente, il tempo medio di `BucketSort` è $\Theta(n)$, mentre nel caso pessimo è $\Theta(n^2)$.

2.9 Problema della selezione

Dato un insieme di elementi A , definiamo formalmente il **problema della selezione**:

- **Input:** Insieme A di n numeri diversi ed un intero i , con $1 \leq i \leq n$.
- **Output:** L'elemento $x \in A$, maggiore di esattamente altri $i - 1$ elementi di A .

In termini pratici, ci concentreremo sulla selezione del valore minimo, massimo e uno particolare dell'insieme. Per i primi due casi è evidente che non sia possibile fare meno di $n-1$ confronti, poiché per determinare un massimo o un minimo è necessario verificare la proprietà per ogni altro elemento dell'insieme. Ne deduciamo quindi che il problema della selezione di uno di questi due valori ha limite asintotico inferiore $\Omega(n)$.

Algorithm 13 findMin(A, n)

Require: A : Insieme degli elementi, n dimensione dell'insieme

```

 $min \leftarrow A[1]$ 
for  $i \leftarrow 2$  to  $n$  do
    if  $min > A[i]$  then
         $min \leftarrow A[i]$ 
    end if
end for
return  $min$ 

```

Se si volesse ricercare simultaneamente il minimo e il massimo sarebbe possibile eseguire questo algoritmo in istanze separate, ottenendo una complessità di $(n-1) + (n-1) = 2n-2 \implies \Theta(n)$ confronti, asintoticamente ottimale. Tuttavia possiamo ridurre ulteriormente il valore dei termini costanti, eseguendo al più $3\lfloor n/2 \rfloor$ confronti.

Ciò si ottiene mantenendo aggiornati il minimo e il massimo con i valori dell'istante corrente. Inizialmente, si confrontano i valori di input, per poi confrontare il più piccolo col minimo corrente ed il più grande con il massimo corrente. Con questo procedimento eliminiamo quegli elementi che hanno "perso" il confronto. L'impostazione dei valori iniziali di massimo e minimo dipende dal numero totale di elementi, in particolare:

- **Dimensione dispari:** Il valore del primo elemento è assegnato al minimo e massimo corrente. Gli altri elementi saranno esaminati a coppie.
- **Dimensione pari:** Si effettua un confronto fra i primi due elementi, mentre i restanti saranno esaminati a coppie.

Analizzando il numero di confronti effettuati possiamo vedere che con dimensione dispari abbiamo $3\lfloor n/2 \rfloor$ confronti, mentre se è pari, ne eseguiamo uno iniziale, seguito da $3(n-2)/2$ confronti, per un totale di $3n/2 - 2$. In entrambi i casi, dunque, il numero totale di confronti è:

$$T(n) = 3\lfloor n/2 \rfloor = \Theta(n)$$

Il problema della selezione generico presenta il medesimo tempo di esecuzione asintotico: $\Theta(n)$. Una prima idea di algoritmo vede una rielaborazione del quickSort() in tal modo che, dato un elemento perno, l'algoritmo operi solo su un sottoarray della partizione. Ciò impatta drasticamente il tempo di esecuzione, che al posto di essere uguale al quickSort

Algorithm 14 randSelect(A, p, r, i)

Require: A : Array, p : Posizione iniziale, r : Posizione finale, i : Valore del pivot

```

if  $p == r$  then return  $A[p]$ 
end if
 $q \leftarrow \text{randPartition}(A, p, r)$        $\triangleright$  Divido l'array in due parti con un perno casuale
 $k \leftarrow q - p + 1$                    $\triangleright$  Numero elementi a sinistra del perno
if  $i == k$  then return  $A[q]$            $\triangleright$  Valore del pivot è soluzione
else if  $i < k$  then return randSelect( $A, p, q-1, i$ )     $\triangleright$  decrementa sezione bassa
else return randSelect( $A, q+1, r, i-k$ )               $\triangleright$  Decrementa sezione alta
end if

```

è ottimale, $\Theta(n)$, nell'ipotesi che gli elementi siano tutti diversi. Come visto nelle sezioni precedenti, si prende un pivot che prenderà la prima posizione del sottoarray della sezione alta. Esiste tuttavia anche un algoritmo deterministico per risolvere il problema delle selezioni in tempo lineare, ma in quanto il suo tempo di esecuzione ha una forte dipendenza dalla quantità delle costanti è difficile poterlo considerare in termini asintotici. In particolare, con grandi numeri è possibile che sia più efficiente un algoritmo nell'ordine di $n \log(n)$.

Questo algoritmo è detto **mediana dei mediani** e si pone lo scopo di scegliere un perno buono deterministicamente, per rimanere in una zona proporzionale. Intuitivamente, segue questi passi:

1. Dividi l'array in gruppi di 5 elementi. Complessità: 0.
2. Calcola il mediano di ogni gruppo. Complessità: $5n$.
3. Calcola ricorsivamente il mediano dei mediani x . Complessità: $T(n/5)$, con $n/5$ approssimato per eccesso. Eccesso a causa del gruppo con gli ultimi elementi restanti.
4. Partiziona rispetto a x , il perno. Complessità: n .
5. Ricorri sulla parte interessata.

Capitolo 3

Strutture dati

3.1 Pile

Le **pile**, o stack, sono insiemi dinamici dove la posizione di un elemento inserito o rimosso è predeterminata. In particolare, questa struttura dati implementa la logica **LIFO**: Last-in, First-out.

L'operazione di inserimento è chiamata **push()** e pone un elemento in cima alla pila; quella di rimozione è invece detta **pop()** e rimuove l'elemento inserito più recentemente. Si presentano inoltre i campi $S.top$, l'indice della cima, e $S.size$, la dimensione della pila. È possibile implementare questa struttura dati su un array di dimensione $S[1 : S.top]$.

Se la stack è vuota, verificabile con la procedura **isEmpty()**, allora $S.size = 0$ e se si cerca di estrarne un elemento, si avrà un **underflow**, mentre se si cerca di inserire un elemento in una pila piena, quindi $S.top = S.size$, si avrà un **overflow**. Inoltre, la logica LIFO può essere descritta tramite sequenze di push tranne per i casi in cui si presenta un errore. Infatti, definendo un'operazione **top()** per recuperare l'elemento in cima alla pila abbiamo:

$$\begin{aligned} Pop(Push(S, x)) &= S, & Top(Push(S, x)) &= x \\ Top(Empty(S)) &= ERR, & Pop(Empty(S)) &= ERR \end{aligned}$$

In ogni caso, ognuna delle tre operazioni è eseguita in tempo costante $O(1)$.

Algorithm 15 isEmpty(S)

```
Require:  $S$ : La pila  
  if  $S.top = 0$  then  
    return TRUE  
  end if  
  return FALSE
```

Algorithm 16 push(S,x)

Require: S : La pila, x : Elemento da inserire

```

if  $S.top = S.size$  then
    error "overflow"
else  $S.top \leftarrow S.top + 1$ 
     $S[S.top] \leftarrow x$ 
end if

```

Algorithm 17 pop(S)

Require: S : La pila

```

if isEmpty(S) then
    error "underflow"
else  $S.top \leftarrow S.top - 1$ 
    return  $S[S.top + 1]$ 
end if

```

3.2 Code con priorità

Le **code**, come le pile, sono un insieme dinamico di elementi con posizioni predeterminate. La differenza principale è l'implementazione della logica **FIFO**, First-in, First-out. Ciò significa che gli oggetti sono inseriti nella struttura all'ultima posizione con **enqueue()** e prelevati a partire dalla cima con **dequeue()**. Si possono implementare con un array $Q[1 : n]$, il quale avrà gli attributi $Q.size$ per la dimensione, mentre $Q.head$ e $Q.tail$ sono usati per puntare al primo e all'ultimo elemento rispettivamente.

All'inizio, con la coda vuota, i campi di testa e coda sono uguali; provando a usare **dequeue()** si avrà un errore di underflow, mentre se si presenta una di queste due istanze:

$$Q.head = Q.tail + 1 \quad \vee \quad (Q.head = 1 \wedge Q.tail = Q.size)$$

La coda sarà piena, e invocando **enqueue()** si avrà un errore di overflow. Inoltre, come per le pile, le seguenti operazioni hanno tempo di esecuzione costante $O(1)$:

Algorithm 18 enqueue(Q, x)

Require: Q : La coda, x : L'elemento da incodare

```
if  $Q.head = Q.tail + 1$  OR ( $Q.head = 1$  AND  $Q.tail = Q.size$ ) then
    error "overflow"
end if
 $Q[Q.tail] \leftarrow x$  ▷  $x$  è ora l'elemento in ultima posizione
if  $Q.tail = Q.size$  then
     $Q.tail \leftarrow 1$ 
else  $Q.tail \leftarrow Q.tail + 1$ 
end if
```

Algorithm 19 dequeue(Q)

Require: Q : La coda

```
if  $Q.head = Q.tail$  then
    error "underflow"
end if
 $x \leftarrow Q[Q.head]$  ▷ Prendi l'elemento in cima
if  $Q.head = Q.size$  then
     $Q.head \leftarrow 1$ 
else  $Q.head \leftarrow Q.head + 1$ 
end if
return  $x$ 
```

3.3 Liste concatenate

3.4 Alberi binari

3.4.1 Alberi di ricerca

3.4.2 RB-alberi

3.4.3 B-alberi binomiali

3.5 Estensione di strutture dati

3.6 Heap binomiali

3.7 Insiemi disgiunti

3.8 Tabelle hash

Capitolo 4

Algoritmi per grafi

4.1 Grafi

Un **grafo** è una struttura usata per rappresentare relazioni tra oggetti, i quali sono rappresentati da **nodi** (vertices), mentre le relazioni tra di essi sono rappresentate da **archi** (edges). È un concetto versatile abbastanza da poter rappresentare qualunque cosa possa essere messa in relazione, come per esempio una rete stradale che collega le città oppure gli stati di una FSM.

Diciamo inoltre **cammino** una sequenza di nodi collegati dagli archi e spesso i problemi trattano la ricerca del cammino **ottimo**, la cui definizione varia in base alla richiesta. Ciò implica la presenza di un qualche tipo di misura; chiamiamo intuitivamente **lunghezza** di un cammino il numero di archi attraversati in un percorso. Ci sono inoltre più tipi di grafo:

- **Grafo orientato:** Grafo i cui cammini sono orientati in una direzione. Presenta un nodo **sorgente** ed uno di **destinazione**. È possibile inoltre avere archi che ritornano sullo stesso nodo, ed in tal caso vale come passo.
- **Grafo non orientato:** Caso speciale del tipo precedente, i cammini possono andare in ambo le direzioni.
- **Grafo pesato:** Grafo i cui archi hanno associato un valore matematico, detto **peso**. Il costo totale è dato dalla somma dei valori degli archi attraversati.

Queste erano tutte definizioni intuitive, ma è necessario garantire formalità per poter lavorare con queste rappresentazioni. Definiamo formalmente un grafo come la coppia:

$$G = (V, E)$$

Dove V è l'**insieme dei nodi** ed E rappresenta quello degli **archi**. In particolare, per i grafi orientati, quest'ultimo è una coppia ordinata di vertici: $E \subseteq V \times V$. In

questo senso un grafo può essere visto anche come una relazione binaria sull'insieme dei vertici, connettendo la struttura con le relazioni e alle matrici. Per esempio, potremmo definire un grafo simmetrico grazie al concetto di relazione simmetrica e otterremmo la dimostrazione di un grafo non orientato.

Definiamo ora formalmente il concetto di cammino: una sequenza di nodi tali per cui per ogni coppia di nodi ci sia un arco che li connette, quindi:

$$v_0, \dots, v_n \text{ tale che } \forall i < n \wedge i \geq 0 (v_i, v_{i+1}) \in E$$

In questi termini possiamo intendere la lunghezza di un cammino come la distanza fra due nodi, definita come il numero totale di nodi meno 1. Un cammino può contenere nodi ripetuti; in particolare, se inizia e termina nello stesso nodo si parla di **ciclo**. Un grafo che contiene almeno un ciclo è detto **ciclico**, altrimenti è **aciclico**.

Generalmente, vale la regola che se un grafo ha lunghezza maggiore del suo numero di nodi distinti, avrà sicuramente almeno un cammino ciclico. Parliamo adesso di **grado** dei nodi; ne esistono due tipi:

- **Grado uscente:** Numero di archi che escono dal nodo, formalmente la cardinalità di insieme v tale per cui la coppia (u, v) appartiene ad E :

$$outDeg(u) = |\{v | (u, v) \in E\}|$$

- **Grado entrante:** Numero di archi che entrano nel nodo, formalmente la cardinalità di insieme u tale per cui la coppia (u, v) appartiene ad E :

$$inDeg(v) = |\{u | (u, v) \in E\}|$$

- **Grado totale:** Somma dei due valori precedenti:

$$inDeg + outDeg$$

Altri concetti importanti sono il **diametro** di un grafo, che indica la massima distanza del cammino minimo tra due nodi, definita come:

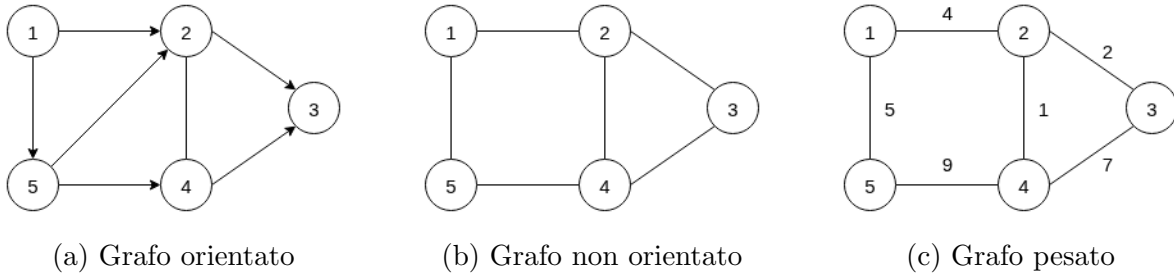
$$diam(G) = \max(dst(u, v))$$

I **sottografi**, intuitivamente un insieme di nodi più piccolo o uguale ottenuto a partire da un altro grafo, definiti come:

$$G' = (V', E') \text{ tale che } V' \subseteq V \wedge E' \subseteq E$$

I **grafi completi** vedono invece ogni coppia di nodi collegata da un arco, la cui definizione differisce in base all'orientazione:

$$\text{Non orientato: } \frac{n(n-1)}{2} \quad \text{Orientato: } n(n-1)$$



Esistono due tecniche principali per rappresentare i grafi in memoria: le **liste di adiacenza** e le **matrici di adiacenza**. Le prime utilizzano un array indicizzato sui nodi, dove ogni posizione contiene la lista dei nodi adiacenti, formalmente:

$$Adj[u] = \{v | (u, v) \in E\}$$

La rappresentazione memorizza esclusivamente gli archi presenti nel grafo, risultando particolarmente efficiente in caso di grafi sparsi. Richiede uno spazio proporzionale alla somma di nodi e archi:

$$\Theta(|V| + |E|), \text{ con abuso di notazione: } \Theta(V + E)$$

Nel caso di grafi pesati, ogni elemento della lista ha un campo per contenere il peso dell'arco.

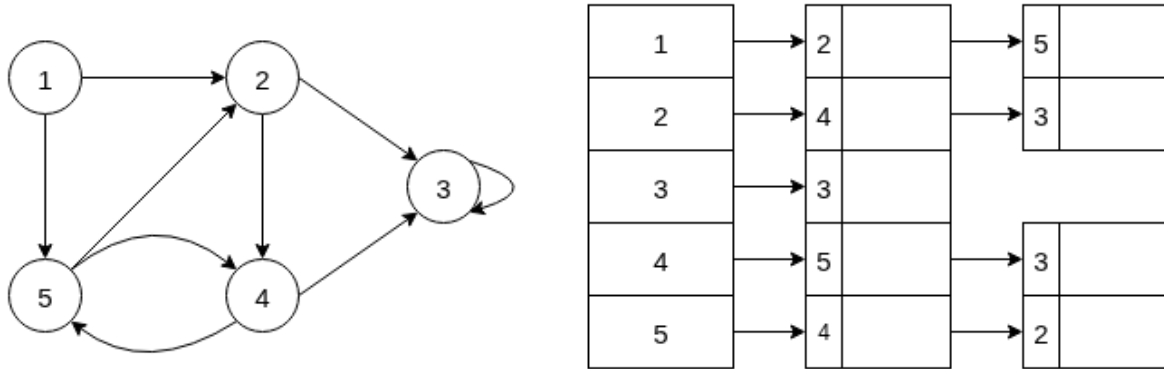


Figura 4.2: Lista di adiacenza

Le matrici di adiacenza, invece, utilizzano una matrice di dimensione $V \times V$ con valori booleani per segnalare le adiacenze per ogni nodo. Formalmente:

$$G[i, j] = \begin{cases} 1 & \text{Se esiste l'arco } (i, j) \\ 0 & \text{altrimenti} \end{cases}$$

Nei grafi non orientati la matrice risulta **simmetrica** e lo spazio richiesto è $\Theta(V^2)$. È una rappresentazione inefficiente per grafi **sparsi** ($|E|$ è molto più piccolo di $|V|^2$), mentre per quelli completi risulta molto comoda. Per i grafi pesati, invece, al posto di valori booleani si utilizza direttamente il peso degli archi e l'assenza di un arco è segnalata con un valore sentinella, un infinito, oppure direttamente un NULL. Facciamo ora un

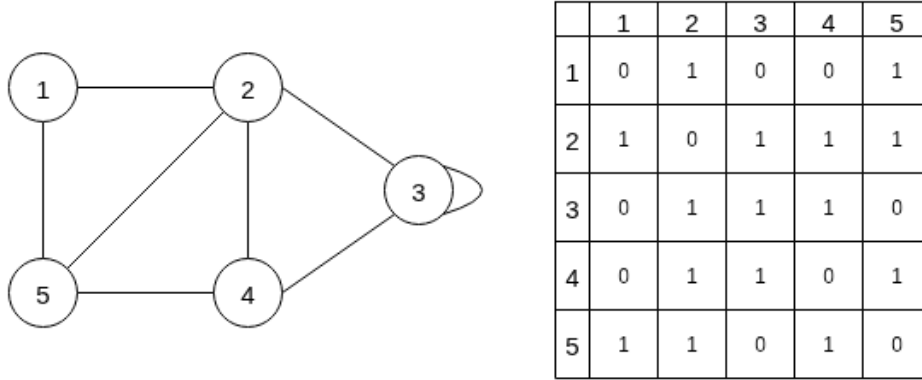


Figura 4.3: Matrice di adiacenza

confronto fra le due rappresentazioni analizzando le modalità di calcolo del grado uscente. Con le liste bisogna contare ogni arco; nel caso pessimo abbiamo una complessità $\Theta(V)$, perché può essere che un nodo sia collegato a tutti gli altri. Alternativamente sarebbe possibile estendere la struttura dati con un campo size, garantendo un accesso a tempo costante. Per il grado entrante bisogna scorrere l'intero grafo, oppure si può calcolare

Algorithm 20 outDeg((v,e), u)

Require: (v, e) : coppia nodi-archi, u : Nodo il cui grado è calcolato

$r \leftarrow 0$

for all $v \in Adj[u]$ **do**

▷ Per ogni nodo adiacente a u

$r \leftarrow r + 1$

▷ Aumenta il counter dei nodi adiacenti

end for

return r

l'outDeg del grafo trasposto, ma costerebbe il doppio della memoria. Per le matrici di adiacenza invece si effettua la sommatoria della riga per il grado uscente e la sommatoria della colonna per quello entrante:

$$outDeg = \sum_{j=1}^n G[i, j] \quad inDeg = \sum_{i=1}^n G[i, j]$$

4.2 Visita in ampiezza

Esistono molti algoritmi per l'esplorazione dei grafi; studieremo i due più utilizzati: la **visita in ampiezza** (BreadthFirstSearch o BFS) e la **visita in profondità** (DepthSearchFirst o DFS), cercando di rispondere a domande riguardo la struttura dei grafi, il cammino minimo e la raggiungibilità. Ci serviremo di un'importante campo per determinare lo stato dei singoli nodi. Diciamo che un nodo è:

- **Bianco** se non è stato esplorato.
- **Grigio** se è stato esplorato, ma non vi è stata effettuata alcuna operazione.
- **Nero** se è stato esplorato e sono state effettuate operazioni.

L'algoritmo BFS è uno dei più semplici per esplorare i grafi e viene utilizzato anche come base per procedure più complesse. Dato un grafo $G = (V, E)$ ed un nodo **sorgente** s , si esplorano gli archi del grafico per scoprire tutti i vertici raggiungibili da s .

Calcola inoltre la distanza fra di esso e ciascun nodo raggiungibile, dove la **distanza** da un vertice v alla sorgente corrisponde al **cammino minimo** fra i due. Nella sua esecuzione genera un albero, detto **Albero BFS** (oppure BF-Tree) e funziona per grafi orientati e non orientati.

- **Caso pessimo:** Temporale: $\Theta(|V| + |E|)$, Spaziale: $\Theta(|V|)$
- **Caso ottimo:** Temporale: $\Theta(|V| + |E|)$
- **Caso medio:** Temporale: $\Theta(|V| + |E|)$

Intuitivamente, una visita in ampiezza espande la frontiera dei vertici scoperti in maniera uniforme; partendo dalla sorgente s , scopre i suoi vicini a distanza 1, poi a distanza 2, e così via finché non trova tutti i vertici raggiungibili dalla sorgente.

In necessità di ricavare i vertici di un cammino minimo da s a v , supponendo sia già stata eseguita la BFS, è possibile utilizzare l'algoritmo **printPath** indicato alla pagina seguente.

Analizziamo ora le varie sezioni dell'algoritmo per ricavarne il tempo di esecuzione. In primo luogo, i vertici sono colorati di bianco esclusivamente nell'inizializzazione e mai più dopo. La verifica di questo colore per ogni nodo nella lista di adiacenza garantisce che ogni nodo sia inserito e rimosso dalla coda al più una volta. Le operazioni di inserimento e cancellazione si eseguono in tempo costante, per un totale di $O(V)$.

La somma delle lunghezze di tutte le liste di adiacenza è data da $\Theta(E)$, ed il tempo per visitarle tutte è pari a $O(V + E)$. In quanto il costo di inizializzazione è entro questo ordine di grandezza, possiamo concludere che il **tempo totale di esecuzione** per l'algoritmo BFS è di $O(V + E)$. Per la dimostrazione del funzionamento dell'algoritmo è necessario definire i seguenti lemmi:

Algorithm 21 BFS(G, s)**Require:** G : Grafo, s Nodo sorgente

```

for all  $u \in V[G]$  do                                ▷ Per ogni nodo del grafo
     $color[u] \leftarrow white$                             ▷ Inizializza il nodo come non visitato
     $d[u] \leftarrow \infty$                                 ▷ Distanza dalla sorgente del nodo
     $\pi[u] \leftarrow NIL$                                 ▷ Predecessore nel BF-Tree
end for

 $color[s] \leftarrow gray$                                 ▷ Nodo sorgente esplorato
 $d[s] \leftarrow 0$                                        ▷ La distanza fra  $s$  ed  $s$  è 0
 $\pi[s] \leftarrow NIL$                                 ▷ La sorgente non ha predecessori
 $Q \leftarrow NIL$                                        ▷ Coda fifo dei nodi grigi
enqueue( $Q, s$ )

while  $Q \neq NIL$  do                                ▷ Finché ci sono nodi grigi
     $u \leftarrow dequeue(Q)$                             ▷ Prendi il primo nodo della coda
    for all  $v \in Adj[u]$  do                            ▷ Ogni nodo in lista di adiacenza di  $u$ 
        if  $color[v] = white$  then                    ▷ Nodo non esplorato?
             $color[v] \leftarrow gray$                 ▷ Nodo ora esplorato
             $d[v] \leftarrow d[u] + 1$                 ▷ La cui distanza è quella del precedente nodo ( $u$ ) più uno
             $\pi[v] \leftarrow u$                         ▷ Predecessore è  $u$ 
            enqueue( $Q, v$ )                            ▷ Nodo ora nella lista di quelli da elaborare
        end if
    end for
     $color[u] \leftarrow black$                             ▷ Il nodo è stato elaborato
end while

```

Lemma 1. Sia il grafo $G = (V, E)$ e $s \in V$ un nodo arbitrario. Allora, per qualsiasi arco $(u, v) \in E$ si ha:

$$\delta(s, v) \leq \delta(s, u) + 1$$

Dunque per ogni coppia di nodi, il cammino minimo fra i nodi s, v annotato come $\delta(s, v)$ non può essere più lungo di quello da s ad u , seguito dall'arco u, v . Se u non è raggiungibile, la distanza è infinita, mantenendo la validità della disuguaglianza.

Lemma 2. Sia $G = (V, E)$ un grafo e supponiamo che venga eseguita BFS a partire da un sorgente $s \in V$. Allora per ogni vertice $v \in V$, il valore di distanza $d[v]$ calcolato dall'algoritmo soddisfa sempre la seguente relazione, anche al termine della procedura:

$$d[v] \geq \delta(s, v)$$

Algorithm 22 printPath(G, s, v)

Require: G : Grafo, s, v : Due nodi

```

if  $v = s$  then
    print( $s$ )
else if  $\pi[v] = NIL$  then
    print(Nessun cammino da  $s$  a  $v$ )
else
    printPath( $G, s, \pi[v]$ )
    print( $v$ )
end if

```

Intuitivamente, è possibile notare che il lemma è vero, poiché ogni valore assegnato a $d[v]$ è uguale al numero di archi di qualche cammino da s a v . Per dimostrarlo formalmente, invece, è necessario ragionare per induzione sui passi eseguiti dalla funzione enqueue. Supponendo come ipotesi induttiva la relazione del lemma, abbiamo:

- **Caso base:** Inserimento della sorgente nella coda. L'ipotesi è vera, perché:

$$d[s] = 0 = \delta(s, s); \quad d[v] = \infty \geq \delta(s, v) \quad \forall v \in V - \{s\}$$

- **Caso induttivo:** Consideriamo un vertice bianco v , scoperto nella visita a partire da un nodo u . L'ipotesi induttiva implica che $d[u] \geq \delta(s, u)$ e grazie al lemma precedente otteniamo:

$$\begin{aligned}
 d[v] &= d[u] + 1 \\
 &\geq \delta(s, u) + 1 \\
 &\geq \delta(s, v)
 \end{aligned} \tag{4.1}$$

Per dimostrare quest'ultima relazione è necessario capire meglio come opera la coda durante l'esecuzione della BFS. Il terzo lemma dice che in qualunque istante, la coda è sempre ordinata per una distanza non decrescente e contiene al più due valori:

Lemma 3. *Supponiamo l'esecuzione di una BFS su un grafo $G = (V, E)$, con la coda Q contenente i vertici $\langle v_1, v_2, \dots, v_r \rangle$, dove v_1 è il primo elemento della coda e v_r l'ultimo. Allora:*

$$d[v_r] \leq d[v_1] + 1 \text{ e } d[v_i] \leq d[v_{i+1}], \text{ con } i = 1, 2, \dots, r - 1$$

Anche la dimostrazione di questo lemma avviene tramite induzione, ma questa volta sul numero di operazioni eseguite sulla coda. Inizialmente, quando questa contiene solo la sorgente, il lemma è banalmente valido, mentre per il passo induttivo è necessario provare che è valido dopo inserimento ed eliminazione di un nodo nella coda.

Per l'eliminazione di un nodo, quando la cima v_1 è eliminata, il successivo nodo v_2 diventa la nuova testa¹ e per ipotesi induttiva abbiamo:

$$d[v_1] \leq d[v_2] \implies d[v_r] \leq d[v_1] + 1 \leq d[v_2] + 1$$

Dunque rimuovere il primo elemento non rompe l'ordine e le restanti disuguaglianze rimangono inalterate. Il lemma è dimostrato con v_2 in cima alla coda.

Per l'inserimento di un nodo, invece, consideriamo la riga dove è invocato il metodo `enqueue()`; quando si inserisce il nodo in una coda, questo diventa v_{r+1} ². Attualmente il nodo in cima alla coda è $u = v_1$ e vale l'ipotesi per cui:

$$d[u] \leq d[v_2]; \quad d[v_r] \leq d[u] + 1$$

Con una coda non vuota, gli si rimuove u per elaborare la sua lista di adiacenza, ma così facendo, il nodo v_2 diventa la nuova cima v_1 della coda, facendo valere:

$$d[u] \leq d[v_1] \implies d[v_{r+1}] = d[v] = d[u] + 1 \leq d[v_1] + 1$$

Siccome $d[v_r] \leq d[u] + 1$, abbiamo che $d[v_r] \leq d[u] + 1 = d[v] = d[v_{r+1}]$ e le restanti disuguaglianze restano inalterate, facendo valere il lemma per l'inserimento di un nodo v .

Adesso possiamo confermare la correttezza della visita in ampiezza. L'algoritmo calcola i cammini minimi e risolve il problema della raggiungibilità con il seguente teorema completo:

Teorema 2. Correttezza della visita in ampiezza

Sia $G = (V, E)$ un grafo orientato o non orientato e supponiamo venga attuata una BFS a partire dal suo nodo sorgente $s \in V$. Allora, durante la sua esecuzione, BFS scopre tutti i nodi $v \in V$ raggiungibili dalla sorgente e, alla termine della sua esecuzione, viene calcolato il cammino minimo da s a v per ogni nodo del grafo.

$$d[v] = \delta(s, v) \quad \forall v \in V$$

Inoltre, per qualsiasi altro nodo $v \neq s$ raggiungibile da quest'ultimo, uno dei cammini minimi da s a v è un cammino minimo da s a $\pi[v]$, seguito dall'arco $(\pi[v], v)$.

Per dimostrarlo, dividiamo i nodi per la distanza reale, definendo V_k come l'insieme dei nodi a distanza k :

$$V_k = \{v \mid \delta(s, v) = k\}$$

Dunque $V_0 = \{s\}$, V_1 = nodi a livello 1, e così via. Facendo induzione su k , dimostriamo che tutti nodi a questa distanza sono scoperti correttamente dalla BFS, dunque livello per livello. Per ogni nodo $v \in V_k$, esiste infatti un momento in cui:

¹Se v_1 era l'unico nodo, il lemma è banalmente valido.

²Se la coda era vuota e questo è il primo nodo inserito, $r = 1$ ed il lemma è vero banalmente.

- Il nodo v è grigio.
- La distanza k è assegnata a $d[v]$.
- Se il nodo corrente v è diverso dalla sorgente s , il predecessore è $\pi[v] = u \in V_{k-1}$, quindi il nodo di livello superiore rimosso da `dequeue()`.
- Il nodo v è inserito in coda con `enqueue()`.

Come passo base consideriamo $k = 0$, quindi V_0 , il livello della sorgente. In questo caso:

- $d[s] = 0 = k$, corretto.
- s è grigio in quanto sta venendo elaborato, corretto.
- s è attualmente in coda, corretto.

Come passo induttivo supponiamo vera l'ipotesi per $k - 1$ prendendo un nodo $v \in V_k$. Per definizione di distanza minima, esiste un nodo a livello superiore $u \in V_{k-1}$ tale che esista un arco fra i due: $(u, v) \in E$. Per ipotesi induttiva:

- u è scoperto correttamente.
- $d[u] = k - 1$.
- u è inserito in coda.

Quando u è rimosso dalla coda per l'analisi della sua lista di adiacenza, scopro anche il nodo v supposto all'inizio del passo, ottenendo $d[v] = d[u] + 1 = k$, $\pi[v] = u$ e v ottiene il colore grigio, dimostrando il teorema.

Inoltre, per l'ultimo lemma enunciato, possiamo confermare che v non è stato precedentemente scoperto, perché avrebbe una distanza minore di k . Questo è impossibile perché k è già il cammino minimo (quindi la distanza) tra la sorgente e il nodo.

Il risultato di una BFS è un albero radicato nel nodo s e se ci sono nodi non raggiunti, questi avranno distanza infinita da esso e non saranno parte dell'albero. Tuttavia, nulla vieta di eseguire un'altra procedura uguale sui nodi bianchi, creando nuovi alberi. Ciò consente di creare una **foresta** di alberi BFS.

4.3 Visita in profondità

L'algoritmo di **Visita in profondità** (DFS, Depth First Search) esplora gli archi uscenti dal vertice v scoperto più recentemente e che ha ancora nodi non esplorati. La procedura continua finché non sono scoperti tutti i vertici raggiungibili da v . Fatto ciò, l'algoritmo sale di un livello per controllare eventuali altri nodi bianchi. Ripete il procedimento fino alla totale esplorazione del grafo.

Caso pessimo temporale: $\Theta(|V| + |E|)$, caso pessimo spaziale: $\Theta(|V|)$

L'algoritmo forma un **sottografo dei predecessori** G_π che può essere formato da più alberi, in quanto la visita può essere ripetuta da più sorgenti. Comprende sempre tutti i vertici e lo definiamo, insieme agli **archi d'albero**, come:

$$G_\pi = (V, E_\pi); \quad E_\pi = \{(\pi[v], v) | v \in V \text{ e } \pi[v] \neq NIL\}$$

Oltre a creare la foresta, la DFS si serve di due **marcatempo**, per segnare il momento in cui un nodo viene scoperto $d[v]$ e l'istante in cui la visita completa l'ispezione della lista di adiacenze del nodo $f[v]$. Agevolano l'analisi del comportamento dell'algoritmo.

Algorithm 23 DFS(G)

Require: G : Grafo
for all $u \in V[G]$ **do** ▷ Inizializzazione del grafo
 $color[u] \leftarrow white, \quad \pi[u] \leftarrow NIL$
end for
 $time \leftarrow 0$ ▷ Marcatempo globale per assegnare $d[v]$, $f[v]$

for all $u \in V[G]$ **do** ▷ Visita ogni nodo bianco del grafo
 if $color[u] = white$ **then** ▷ Se non è raggiunto, inizia una nuova visita DFS
 $DFS_Visit(G, u)$
 end if
end for

Algorithm 24 DFS_Visit(G, u)

Require: G : Grafo, u : Nodo preso come sorgente
 $time \leftarrow time + 1$ ▷ Aggiorno marcatempo
 $d[u] \leftarrow time, \quad color[u] \leftarrow gray$ ▷ Tempo in cui il nodo è scoperto

for all $v \in Adj[u]$ **do** ▷ Controlla tutti i suoi nodi figli
 if $color[v] = white$ **then** ▷ Se non è esplorato, continua ad andare giù
 $\pi[v] \leftarrow u$
 $DFS_Visit(G, v)$ ▷ Ricorsione fa usare la stack per salvare i nodi
 end if
end for

 $time \leftarrow time + 1$
 $f[u] \leftarrow time, \quad color[u] \leftarrow black$ ▷ Tempo in cui la visita è finita

Prendiamo come esempio un grafo orientato di otto nodi. Scegliamo di invocare la procedura DFS considerando il nodo s come la sorgente. Allora si controllerà ogni nodo nella sua lista di adiacenza fin quando non saranno stati esplorati tutti, segnando tempo di scoperta e tempo di conclusione esplorazione. Al termine della procedura avremo come risultato una foresta di due alberi; uno radicato in s e l'altro in t .

La misura temporale dei nodi è molto importante, perché il tempo di elaborazione del nodo finale corrisponde esattamente al doppio del numero di nodi del grafo. Ciò accade perché si assegnano due misure temporali per nodo.

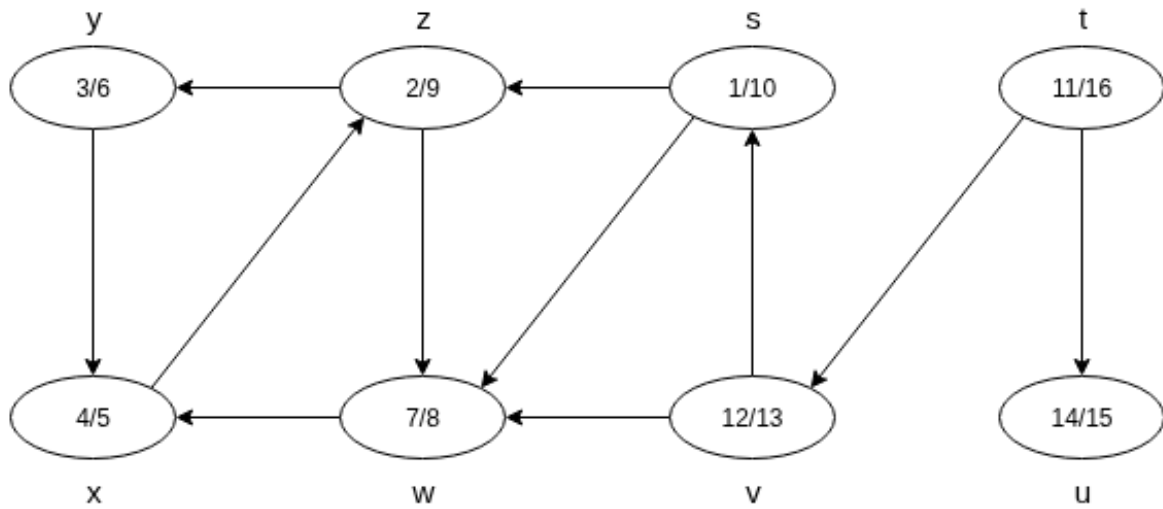


Figura 4.4: Esempio di procedura DFS

Analizziamo ora la complessità dell'algoritmo: i primi due cicli for nel primo algoritmo usano un tempo $\Theta(V)$, poiché sono effettuati sull'intero insieme di nodi del grafo, escludendo la DFS_Visit. Questa è invece chiamata una sola volta per ogni nodo bianco nelle liste di adiacenza, quindi $|Adj[v]|$ volte.

Siccome l'insieme delle liste di adiacenza compone quello degli archi e la visita è chiamata una sola volta per ogni nodo, confermiamo che DFS_Visit ha complessità $\Theta(V + E)$; il tempo di esecuzione totale dell'algoritmo DFS è quindi $\Theta(V + E)$. Enunciamo ora alcuni teoremi importanti per questo algoritmo:

Teorema 3. Teorema delle parentesi

In una qualsiasi visita in profondità di un grafo G orientato o non orientato, per ogni coppia di vertici u, v è soddisfatta esclusivamente una delle seguenti due condizioni:

- *Gli intervalli $[d[u], f[u]]$ e $[d[v], f[v]]$ sono completamente disgiunti. I nodi non discendono l'uno dell'altro.*

- *L'intervallo $[d[u], f[u]]$ o $d[v], f[v]$ è interamente contenuto nell'altro, rendendo il primo nodo discendente del secondo.*

Per dimostrarlo, prendiamo la coppia di nodi u, v e assumiamo, senza perdita di generalità, che valga la seconda relazione. Abbiamo quindi $d[u] < d[v]$ oppure $d[v] < d[u]$. Qui possiamo distinguere due casi:

- Se $f[u] < d[v]$ abbiamo due intervalli disgiunti, dimostrando la prima condizione del teorema.
- Se $d[v] < f[u]$ abbiamo un'intersezione tra gli insiemi, stando dentro la chiamata DFS_Visit del nodo u . Ci troviamo quindi necessariamente in un sottointervallo i cui estremi sono $d[u], f[u]$, dimostrando che è interamente contenuto nel sovraintervallo.

Un'altra questione molto importante riguarda la possibilità di conoscere i discendenti a partire da un nodo sorgente. L'algoritmo DFS è capace di rispondere a questa domanda, servendosi del concetto dei nodi bianchi:

Teorema 4. Teorema del cammino bianco

In una foresta di visita in profondità di un grafo G , orientato o non orientato, il vertice v è discendente dal nodo u se e solo se al tempo $d[u]$ in cui viene scoperto, il vertice v può essere raggiunto dall'altro lungo un cammino formato esclusivamente da nodi bianchi.

Supponiamo infatti due nodi $u, v \in V$, di cui il secondo è discendente dal primo. Sia ora un terzo nodo w nel cammino fra i due nella foresta. Se è bianco, il teorema è banalmente dimostrato perché l'intervallo di w è interamente contenuto in quello di u .

In quanto discendente, il nodo w è necessariamente visitato dopo il padre e sarà per forza bianco, perché l'unico modo per inizializzare il valore $d[w]$ è invocare DFS_Visit, cosa che avviene solamente se il nodo è bianco. Difatti, prima di inizializzare $d[u]$, il nodo w è bianco, e al tempo $d[u]$ esiste un cammino di nodi bianchi che comprende w .

Potremmo assumere ora per assurdo che v non discenda da u ma che abbia come predecessore w , discendente di u . Ne risulta la disuguaglianza:

$$d[u] < d[w] < f[w] < f[u]$$

Allora al tempo $d[w]$ sto visitando i nodi della lista di adiacenza di w , nella quale è presente pure v , che inevitabilmente diventa discendente di u , dimostrando il teorema. Di nuovo.

Durante una visita in profondità possono apparire diversi tipi di archi. Il loro studio consente di ottenere alcune importanti informazioni:

- **Archi d'albero T:** Archi che collegano un intervallo con un suo discendente.

- **Archi all'indietro B:** Archi che collegano un nodo al suo antenato.
- **Archi in avanti F:** Archi che collegano un intervallo con un discendente non diretto.
- **Archi trasversali C:** Archi che collegano due intervalli disgiunti.

In particolare, durante una visita in profondità è possibile riconoscere il tipo di arco in base al colore dei nodi: il bianco indica un arco d'albero, il grigio uno all'indietro, mentre il nero un arco in avanti o trasversale. Inoltre, se un grafo presenta archi all'indietro è sicuramente **ciclico**, perché, supponendo che la visita in profondità ne trovi uno, v avrà colore grigio e u sarà suo discendente. Esisterà di conseguenza un cammino tra i due nodi.

Supponiamo ora un grafo ciclico e prendiamo in esame un suo ciclo qualsiasi. Sia u il suo primo nodo visitato in profondità e v il predecessore di u nel ciclo. Allora al tempo $d[v]$ tutti i nodi del ciclo tranne u sono bianchi, rendendo il nodo v raggiungibile a partire dall'altro con un cammino di soli nodi bianchi. In quanto tale, possiamo dire per il teorema del cammino bianco che v discende da u nella foresta e sarà visitato quando u è grigio, dimostrando l'arco all'indietro.

4.4 Ordinamento topologico

Definiamo un **ordinamento topologico** di un grafo orientato aciclico (DAG) un ordinamento totale di tutti i suoi nodi, tale che, se G contiene un arco (u, v) , allora u appare prima di v . Possiamo immaginarlo come una distensione dei vertici lungo una linea orizzontale, in modo che tutti gli archi siano direzionati da sinistra a destra; con questi presupposti, non è un concetto applicabile a grafi ciclici. Intuitivamente, l'algoritmo sfrutta la visita in profondità e la stack di attivazione per salvare su di essa ogni nodo quando è colorato di nero. Eseguendo operazioni di pop, è possibile ottenere il grafo ordinato topologicamente.

Algorithm 25 topologicalSort(G)

Require: G : Grafo orientato aciclico.

Chiama $DFS(G)$ per calcolare $f[v]$ di ogni nodo del grafo

Completata la visita di un nodo, esegui $push(v)$

Elaborati tutti i nodi, esegui $pop()$ per ogni nodo in stack

È immediato capire che è possibile inserire i nodi ordinati in una qualunque struttura dati come una lista concatenata; basta semplicemente eseguire l'operazione di inserimento sul nodo rimosso dalla stack.

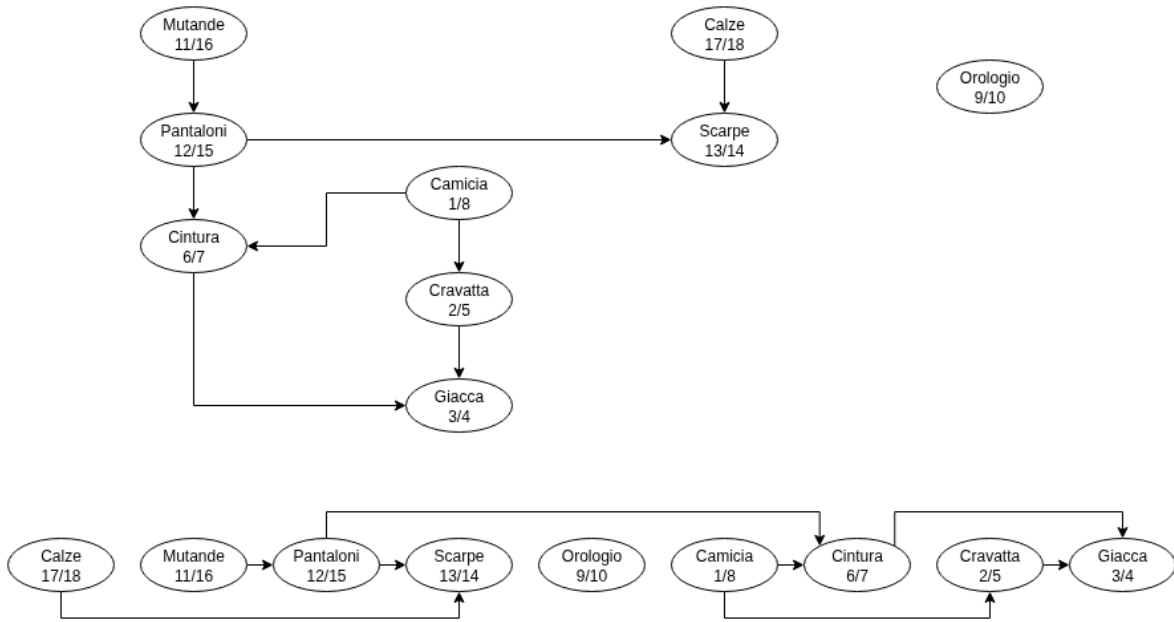


Figura 4.5: Visualizzazione ordinamento topologico

L'algoritmo richiede un tempo $\Theta(V + E)$, perché è quello richiesto dalla visita in profondità. Inoltre, occorre un tempo costante $O(1)$ per rimuovere un totale di $|V|$ nodi dalla stack di attivazione. Per dimostrarlo, è in primo luogo necessario assicurarsi che il grafo passato per parametro sia aciclico grazie al seguente lemma:

Lemma 4. Aciclicità di un grafo

Un grafo orientato G è aciclico se e solo se in una sua visita in profondità non sono prodotti archi all'indietro.

Supponiamo che eseguendo una DFS si produca un arco all'indietro (u, v) . Inevitabilmente, il nodo v sarà un antenato di u nella foresta DFS. Esisterà quindi un cammino nel grafo che va da u a v e l'arco all'indietro (u, v) completa un ciclo. Potendo distinguere i tipi di grafo, possiamo enunciare il teorema:

Teorema 5. Ordinamento topologico

L'algoritmo `topologicalSort()` produce un ordinamento topologico del grafo orientato aciclico che gli viene fornito in input.

Sappiamo che in questo algoritmo è invocata una visita in profondità su un grafo G per calcolare il tempo di elaborazione di ogni suo nodo. Per dimostrare il teorema, è sufficiente provare che per una qualunque coppia di vertici distinti $u, v \in V$, se esiste un arco del grafo che va da u a v , allora vale $f[v] < f[u]$.

Consideriamo quindi un arco qualsiasi; quando è visitato il vertice v non può essere grigio, perché sarebbe antenato di u e costituirebbe un arco all'indietro, rompendo il lemma precedente. Quindi, per qualunque arco, ci troviamo sicuramente in uno di questi due casi:

- Il nodo v è bianco: v è discendente di u e vale $f[v] < f[u]$
- Il nodo v è nero: la visita di v è già stata completata e ha il valore $f[v]$ già impostato prima di $f[u]$. Dunque, certamente minore.

Un'altra comune applicazione della visita in profondità è la **scomposizione** di un grafo orientato nelle sue **componenti fortemente connesse** (SCC), il cui insieme è definito come il numero massimale di vertici $C \subseteq V$ tale che per ogni coppia di nodi $u, v \in C$, si ha che questi sono mutualmente raggiungibili.

In presenza di un grafo non orientato si potrebbe eseguire l'unione di tutti gli insiemi separati con componenti mutualmente raggiungibili, con il seguente algoritmo:

Algorithm 26 ConnectedComponents(G)

Require: G : Grafo aciclico non orientato

```

for all  $v \in V[G]$  do
    makeSet( $v$ )
end for
for all  $(u, v) \in E[G]$  do
    union( $u, v$ )
end for
```

Richiede un tempo lineare per l'esecuzione: $\Theta(V + E)$, tuttavia non è utilizzabile con grafi orientati, per i quali è richiesta una raggiungibilità **bidirezionale**. L'algoritmo per le componenti fortemente connesse, infatti, utilizza due visite in profondità; la prima per calcolare il tempo di elaborazione dei nodi del grafo, la seconda fa lo stesso, ma sul grafo trasposto G^T , definito come $G^T = (V, E^T)$, dove $E^T = \{(u, v) : (v, u) \in E\}$, quindi con ogni arco inverso rispetto a quelli del principale grafo.

G e G^T hanno esattamente le stesse componenti fortemente connesse: due nodi sono raggiungibili l'uno dall'altro nel primo grafo se e solo se ciò vale anche nel trasposto. L'idea alla base dell'algoritmo deriva da una proprietà fondamentale del **grafo delle componenti** $SCC = (V^{SCC}, E^{SCC})$: supponiamo che un grafo abbia componenti fortemente connesse $C_1, \dots, C_k$ e che l'insieme dei vertici $V^{SCC} = \{v_1, \dots, v_k\}$ contiene un vertice v_i per ogni C_i del grafo. Esiste poi un arco $(v_i, v_j) \in E^{SCC}$ se G contiene un arco orientato (x, y) per qualche $x \in C_i$ e qualche $y \in C_j$.

Insomma, contraendo tutti gli archi i cui vertici incidenti sono all'interno della stessa componente fortemente connessa del grafo sicché rimanga un solo vertice, si ottiene

Algorithm 27 SCC(G)**Require:** G : Grafo aciclico orientatoChiama DFS(G) per calcolare $f[u]$ di ogni nodo u del grafoCrea il grafo trasposto G^T Chiama DFS(G^T) considerando i nodi in ordine decrescente rispetto a $f[u]$

Ritorna i nodi di ogni albero della foresta DFS come una singola SCC

il grafo G^{SCC} . Banalmente, raccogli le componenti mutualmente raggiungibili in un sottoinsieme e lo consideri componente fortemente connessa C .

La procedura richiede un tempo lineare per il calcolo delle SCC, quindi $\Theta(V + E)$ e per dimostrarla è necessario enunciare i seguenti lemmi:

Lemma 5. Distinzione delle componenti

Siano C, C' due componenti fortemente connesse distinte in un grafo orientato G . Se $u, v \in C$ e $u', v' \in C'$, e supponendo ci sia un cammino che collega u e u' , allora non può esistere anche un cammino da v' a v nello stesso grafo.

Il lemma è valido per la distinzione delle componenti, perché se esiste un cammino $v' \rightsquigarrow v$ nel grafo, allora esistono i cammini $u \rightsquigarrow u' \rightsquigarrow v'$ e $v' \rightsquigarrow v \rightsquigarrow u$, rendendo u e v raggiungibili mutualmente, contraddicendo l'ipotesi che C e C' siano SCC distinte.

Lemma 6. Tempo di elaborazione

Siano C, C' due componenti fortemente connesse distinte in un grafo orientato. Supponiamo l'esistenza di un arco $(u, v) \in E$, dove $u \in C'$ e $v \in C$. Allora il tempo di fine visita di C' è maggiore di C : $f(C') > f(C)$.

Per la dimostrazione è necessario considerare due casi, in base a quale SCC ha il nodo scoperto prima durante la visita DFS:

- Se $d(C') < d(C)$, diciamo x il primo vertice scoperto in C' ; quindi al tempo $d[x]$ tutti gli altri nodi sono bianchi e nel grafo esiste un cammino da x a tutti gli altri composto da soli nodi bianchi, facendo valere il relativo teorema.

In quanto tale, tutti i vertici di C e C' diventano discendenti di x nell'albero di visita DFS e questo nodo avrà tempo di finitura massimo rispetto a tutti i suoi discendenti: $f[x] = f(C') > f(C)$.

- Se $d(C') > d(C)$, prendiamo y primo vertice scoperto in C per cui $d[y] = d(C)$, al cui tempo tutti i nodi in C sono bianchi ed il grafo comprende un cammino da y a tutti gli altri nodi in C , facendo valere il teorema del cammino bianco.

In quanto tale, tutti i nodi di C diventano discendenti di y nell'albero della visita DFS e, per l'annidamento degli intervalli dei discendenti, vale $f[y] = f(C)$.

Sappiamo inoltre che $d(C') < d(C) = d[y]$, al cui tempo tutti i vertici in C' sono bianchi. Siccome esiste un arco da C' a C , per rispettare la condizione che distingue le SCC, non esisterà un arco da C a C' , rendendo tutti i vertici in quest'ultimo irraggiungibili da y . Dunque, al tempo $f[y]$ i vertici di C' sono ancora bianchi, dimostrando che per qualunque vertice di C' vale $f(C') > f(C)$.

Come conseguenza diretta, per garantire che le SCC rimangano distinte, il grafo trasposto non formerà nuovi archi per creare nuove componenti fortemente connesse. Abbiamo strumenti a sufficienza per la dimostrazione del teorema di questo algoritmo:

Teorema 6. Componenti fortemente connesse

La procedura $SCC()$ calcola correttamente le componenti fortemente connesse di un grafo orientato G .

La dimostrazione avviene per induzione sul numero di alberi creati dalla visita DFS sul grafo G^T . L'ipotesi è che i primi k alberi formati siano componenti fortemente connesse. Quando $k = 0$, banalmente, non avendo componenti, il teorema vale.

Nel passo induttivo supponiamo l'ipotesi e prendiamo in considerazione l'albero $(k + 1)$ -esimo e supponiamo che la sua radice sia il nodo u , che si trova nella SCC C . Per il funzionamento della visita DFS, al tempo di scoperta della radice, tutti gli altri nodi della SCC sono bianchi e formano un cammino nell'albero; inoltre, il tempo di completamento della radice sarà maggiore rispetto a quello di tutti gli altri nodi suoi discendenti.

Ogni arco in G^T che esce da C deve entrare in altre componenti già visitate, quindi nessun vertice appartenente a componenti diverse da C è un discendente della radice u nell'albero di visita DFS, provando che i vertici dell'albero DFS in G^T formano esattamente una sola SCC, dimostrando il teorema.

4.5 Alberi di copertura di costo minimo

Spesso ci si trova davanti a problemi di ottimizzazione; nella teoria dei grafi si tratta il problema dell'**albero di copertura di costo minimo** (MST). Si parte da un grafo connesso non orientato, i cui archi hanno associato un valore, detto **peso**, che indica il costo richiesto per collegare due nodi. La soluzione di questo problema è un sottoinsieme aciclico del grafo di partenza che collega tutti i vertici, il cui peso totale è minimo.

Gli algoritmi trattati in questa sezione, **kruskal** e **prim**, sono avidi e utilizzano la stessa strategia di base applicata in modi differenti. Iniziamo definendo l'algoritmo per la creazione di un albero di copertura minimo **generic-MST()**, il quale prende in input un grafo connesso non orientato con una funzione per il peso. Costruisce l'albero di connessione minimo un arco alla volta, per poi ritornarlo al termine della sua esecuzione.

Questa procedura generica gestisce quindi un insieme A di archi, assicurandosi che prima di ogni iterazione sia un sottoinsieme di qualche albero di connessione minimo. Per

Algorithm 28 generic-MST(G, w)

Require: G : Grafo connesso non orientato, w : Valore del peso $A = \emptyset$ **while** A non è albero di connessione **do** Cerca un arco (u, v) sicuro per A $A = A \cup \{(u, v)\}$ **end while**return A

dimostrare la correttezza abbiamo bisogno di definire alcune entità di base. Anzitutto, un **taglio** $(U, V - U)$ è una partizione dell'insieme dei nodi V , il quale sarà attraversato da alcuni archi, detti **archi del taglio**. Tra questi, quello con il peso minore è detto **arco sicuro** e sarà quello scelto per la costruzione dell'MST.

Teorema 7. Correttezza di generic-MST()

Sia (u, v) un arco sicuro per un taglio $(U, V - U)$ in un grafo G , allora esiste un albero di copertura di costo minimo T che contiene (u, v) .

Banalmente, se l'arco sicuro (u, v) è all'interno di T , la tesi è dimostrata. Alternativamente, è possibile notare che, essendo il grafo connesso, esiste almeno un arco (x, y) che attraversa il taglio. Se questo ha il costo minimo tra tutti gli archi del taglio disponibili, è quello sicuro, e sarà unito all'MST. Dunque:

$$T' = T - \{(u, v) \cup (x, y)\}$$

Il grafo T' è ancora un albero di copertura e il suo costo non è maggiore di quello di T . Pertanto T' è un albero di copertura minimo che contiene (u, v) .

Il primo algoritmo trattato è quello di **Kruskal**; trova un arco sicuro da aggiungere alla foresta in costruzione scegliendo l'arco dal peso minimo. Generalmente per la sua implementazione è usata una struttura dati per insiemi disgiunti; all'inizio saranno presenti vari set contenenti i vertici di un albero della foresta corrente. L'operazione di findSet(u) restituirà un rappresentante dell'insieme che contiene il nodo u , quindi per determinare se due vertici appartengono allo stesso insieme, si verifica se il risultato di findSet per entrambi è uguale. Per unire eventualmente gli alberi, si fa uso di union().

All'inizio dell'algoritmo è presente una fase di inizializzazione: si definisce la foresta vuota e si crea un insieme per ogni nodo del grafo. Si crea poi una lista di tutti gli archi del grafo e la si ordina con criterio monotono crescente rispetto al peso degli archi.

Dopodiché è presente un ciclo per la verifica di appartenenza ad uno stesso albero tramite findSet(). In caso affermativo, l'arco dei due nodi non può essere inserito perché è già dentro la foresta e viene quindi ignorato; alternativamente viene unito alla foresta e i nodi vengono fusi.

Algorithm 29 $\text{kruskal}(G, w)$

Require: G : Grafo connesso, w : Peso dell'arco

```

 $A \leftarrow \emptyset$ 
for all  $v \in V[G]$  do  $\text{makeset}(v)$ 
    Crea una lista  $L$  di tutti gli archi del grafo
    Ordina la lista in modo monotono non decrescente rispetto al peso  $w$ 
end for
for all  $(u, v) \in L$  preso nell'ordine della lista do
    if  $\text{findSet}(u) \neq \text{findSet}(v)$  then ▷ Se l'arco non è nella foresta, uniscilo
         $A \leftarrow A \cup \{(u, v)\}$ 
         $\text{union}(u, v)$ 
    end if
end for
return  $A$ 

```

L'algoritmo di Kruskal non fa un taglio, data la richiesta di un grafo connesso abbiamo la garanzia che questo esiste, consentendoci di trovare un arco sicuro per qualche taglio che è garantito esista. L'insieme A che ritorna la procedura è l'albero di copertura di costo minimo.

Il tempo di esecuzione di Kruskal dipende principalmente dalla struttura dati implementata per gli insiemi disgiunti. Consideriamo il costrutto ideale dato dalla **foresta di insiemi disgiunti** per ottenere il tempo asintoticamente minore rispetto ad altre implementazioni. L'inizializzazione richiede un tempo $O(1)$, creare una lista per ogni arco $O(V + E)$, ordinare gli archi $O(E \log(E))$, le operazioni del secondo ciclo **for** sono poi ripetute $O(E)$ volte. Assieme alle $|V|$ operazioni di $\text{makeSet}()$, le operazioni sugli insiemi disgiunti richiedono un tempo totale di:

$$O((V + E)\alpha(V))$$

Con α funzione dalla crescita estremamente lenta. Essendo poi il grafo connesso, possiamo dire che $|E| \geq |V| - 1$ e quindi le operazioni su insiemi disgiunti richiedono tempo $O(E\alpha(V))$. Da qui, osservando che $|E| < |V|^2 \implies \log|E| = O(\log(V))$ possiamo concludere che il tempo di esecuzione totale per l'algoritmo di Kruskal è:

$$O(E \log(E)) \implies O(E \log(V))$$

Un altro algoritmo per il calcolo di MST è quello di **Prim**, la cui proprietà è che gli archi nell'insieme A che costruisce in esecuzione formano sempre un albero singolo. Inizia infatti da un arbitrario nodo r , radice dell'albero, per poi andare a coprire tutti gli altri vertici passando sempre per gli archi dai pesi minimi.

Come input sono richiesti un grafo connesso G e la radice dell'albero r . L'algoritmo è tipicamente implementato mantenendo una coda di priorità minima di tutti i vertici

che non appartengono all'albero, basata su un attributo **key**, il peso minimo di un arco qualsiasi che collega v a un nodo nell'albero. Si indica con $key[v]$. Inoltre, l'algoritmo mantiene implicitamente l'insieme A come:

$$A = \{(v, \pi[v]) : v \in V - \{r\} - Q\}$$

Al termine dell'algoritmo, la coda creata è del tutto vuota ed è creato l'albero di connessione minimo tale che:

$$A = \{(v, \pi[v]) : v \in V - \{r\}\}$$

Algorithm 30 $\text{prim}(G, w, r)$

Require: G : Grafo connesso, w : Peso dell'arco, r : Nodo radice

```

for all  $u \in V[G]$  do                                ▷ Ogni nodo a sé. No predecessore, distanze infinite.
     $key[u] \leftarrow \infty, \pi[u] \leftarrow NIL$ 
end for
 $key[r] \leftarrow 0, Q \leftarrow \emptyset$ 

for all  $u \in V[G]$  do
     $enqueue(Q, u)$ 
end for

while  $Q \neq \emptyset$  do
     $u \leftarrow extractMin(Q)$                                 ▷ Prendo un nodo oltre il taglio
    for all  $v \in Adj[u]$  do
        if  $v \in Q$  AND  $w(u, v) < key[v]$  then                ▷ Se arco leggero è minore del corrente
             $\pi[v] \leftarrow u, key[v] \leftarrow w(u, v)$         ▷ Aggiorno i valori del nodo corrente
             $reduceKey(Q, v, w(u, v))$                         ▷ Riduco la chiave perché trovato peso minore
        end if
    end for
end while

```

Il tempo di esecuzione di Prim, Come per Kruskal, dipende dalla struttura dati utilizzata per implementare la coda. Se Q è un min-heap binario ci sarà richiesta una $buildHeap()$, di costo $O(V)$. Il corpo del while è eseguito $|V|$ volte e siccome ogni estrazione di minimo è eseguita in un tempo $O(\log(V))$, il tempo totale per terminare il ciclo è $O(V \log(V))$. Il ciclo for viene poi eseguito $|E|$ volte complessivamente e il test che verifica l'appartenenza alla coda Q richiede tempo costante. Ogni chiamata a $reduceKey()$ usa infine un tempo $O(\log(V))$. Possiamo concludere che il tempo di esecuzione totale dell'algoritmo è:

$$O(V \log(V) + E \log(V)) = O(E \log(V))$$

4.6 Cammini minimi a sorgente singola

Il problema del **cammino minimo** è intuitivamente la ricerca di un percorso in un grafo da un nodo sorgente a una destinazione, il cui costo totale, dato dalla somma dei pesi degli archi, è minimo. Formalmente, è richiesto un grafo orientato pesato $G = (V, E)$, la cui funzione peso descritta da $w : E \rightarrow \mathbb{R}$ associa un valore ad ogni arco del grafo. Il peso di un cammino sarà dato dall'espressione:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

Mentre il peso di un cammino minimo $\delta(u, v)$ è definito come:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \rightsquigarrow v\} & \text{Se esiste un cammino tra i nodi} \\ \infty & \text{Altrimenti} \end{cases}$$

Dunque un cammino minimo da un nodo u ad un vertice destinazione v è definito come un cammino qualsiasi con peso $w(p) = \delta(u, v)$. In particolare, questo percorso contiene anche i cammini minimi dalla sorgente a tutti i nodi intermedi per arrivare a v , grazie al seguente lemma:

Lemma 7. Sottocammini di cammini minimi

Dato un grafo orientato pesato G con la relativa funzione del peso w , sia un cammino $p = \langle v_0, v_1, \dots, v_k \rangle$ quello minimo da v_0 a v_k . Consideriamo due valori i, j compresi tra 0 e k tali da definire il percorso $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$. Quest'ultimo è un sottocammino di p ed è anche il percorso minimo dal nodo v_i al vertice v_j .

Per il corretto funzionamento degli algoritmi trattati in questa sezione, è necessario rispettare alcune condizioni per garantire la risolubilità del problema preso in esame. Per esempio, supponiamo un grafo i cui pesi possono essere negativi; sarebbe possibile attraversare all'infinito l'arco con tale valore per ridurre il costo del cammino. Ciò rimuove ogni significato della ricerca, e per questa ragione l'algoritmo di Dijkstra contempla esclusivamente grafi pesati con valori non negativi.

Espandiamo il discorso considerando i cicli positivi: i grafi non possono contenere nemmeno questi, perché eliminandoli dal cammino si ottiene un percorso con la stessa sorgente e destinazione con un peso più piccolo. Gli unici cicli contemplati sono quelli il cui attraversamento è a costo zero, i quali si possono direttamente rimuovere dal grafo in quanto ininfluenti.

La **rappresentazione** dei cammini minimi è simile a quella usata per gli alberi BFS. Dato un grafo definiamo un predecessore $\pi[v]$ che può essere un altro nodo o il valore NIL. È impostato in modo tale per cui sia possibile ripercorrere la catena dei predecessori,

ritornando appunto il cammino minimo tramite la procedura **printPath()**. Inoltre, grazie al lemma dei sottocammini, sappiamo che è possibile usare il metodo per ritornare anche i sottocammini minimi. In ogni caso, il prodotto della funzione sarà un **albero di cammini minimi**, il quale conterrà i percorsi dalla sorgente alla destinazione definiti minimi in base al peso degli archi.

Gli algoritmi della sezione usano la tecnica del **rilassamento**: la verifica dell'esistenza di un percorso migliore per la destinazione. Per ogni vertice del grafo è mantenuto un valore $d[v]$ detto **stima del cammino minimo**, il quale è un limite superiore per il peso di un cammino minimo dalla sorgente alla destinazione. Per eseguire calcoli corretti, è necessaria la seguente inizializzazione:

Algorithm 31 initializeSingleSource(G, s)

Require: G : Grafo orientato pesato, s : Nodo sorgente

```

for all  $v \in V[G]$  do
     $d[v] \leftarrow \infty$ 
     $\pi[v] \leftarrow \text{NIL}$ 
end for
 $d[s] = 0$ 

```

Questa procedura, essendo eseguita per ogni nodo del grafo, richiede un tempo di esecuzione $\Theta(V)$. Segue il rilassamento degli archi grazie alla procedura **relax()**, eseguita in tempo $O(1)$:

Algorithm 32 relax(u, v, w)

Require: u : Sorgente, v : Destinazione, w : Peso

```

if  $d[v] > d[u] + w(u, v)$  then                                 $\triangleright$  Se è trovato un percorso migliore
     $d[v] \leftarrow d[u] + w(u, v)$                                  $\triangleright$  Questa diventa la stima del cammino minimo
     $\pi[v] \leftarrow u$ 
end if

```

Il processo di rilassamento continua fino a quando non è più possibile rilassare gli archi; in tal caso è stato trovato il cammino minimo dalla sorgente alla destinazione.

Il primo algoritmo discusso della sezione è quello di **Dijkstra**, il quale, eseguito su un grafo orientato pesato i cui costi degli archi sono tutti non negativi, consente di trovare il cammino minimo da un nodo sorgente s ad una destinazione v e a tutti i nodi intermedi u_i del percorso scelto secondo una strategia avida. Mantiene inoltre un insieme S su vertici i cui pesi finali dei cammini minimi sono già stati determinati; l'algoritmo seleziona infatti ripetutamente il vertice $u \in V - S$ con la stima minima; lo aggiunge a S e rilassa tutti gli archi uscenti dal nodo.

A seguito dell'inizializzazione della rappresentazione dei nodi $O(V)$, della coda e dell'insieme, si inseriscono tutti i nodi del grafo nella coda $O(V)$, dalla quale ne sarà estratto

Algorithm 33 `dijkstra(G, w, s)`

Require: G : Grafo pesato orientato, w : Peso non negativo, s : Sorgente`initializeSingleSource(G, s)` $S \leftarrow \emptyset$ $Q \leftarrow \emptyset$ **for all** $u \in V[G]$ **do**`enqueue(Q, u)` $\triangleright$ Inserisci ogni nodo nella coda**end for****while** $Q \neq \emptyset$ **do** $\triangleright$ Finché la coda non è vuota $u \leftarrow \text{extractMin}(Q)$ $\triangleright$ Estrai il nodo corrente $S \leftarrow S \cup u$ $\triangleright$ Aggiungilo all'insieme**for all** $v \in \text{Adj}[u]$ **do** $\triangleright$ Rilassa ogni arco nella sua lista di adiacenza`relax(u, v, w)`**if** `relax()` diminuisce il valore $d[v]$ **then** $\triangleright$ Se trovato un cammino migliore`reduceKey(Q, v, d[v])` $\triangleright$ Riduci la chiave della coda**end if****end for****end while**

uno alla volta, ad ogni iterazione del ciclo `while`, per un totale di $|V|$ volte. Al suo interno si unisce il nodo estratto nell'insieme S e si rilassano tutti gli archi da esso uscenti aggiornando la stima $d[u]$ e il predecessore $\pi[v]$ se è trovato un cammino meno costoso. Quando ciò accade, si fa una chiamata alla funzione `reduceKey()` per aggiornare la coda. Ciò accade un totale di $|E|$ volte, quindi per ogni arco del grafo.

La complessità dell'algoritmo di Dijkstra dipende tuttavia anche dalla struttura dati utilizzata per la sua implementazione e quindi ragionare sulle procedure di estrazione del minimo e riduzione chiave. In particolare:

- **Heap:** `extractMin()` e `reduceKey()` richiedono entrambe un tempo $\log(n)$, facendo risultare come tempo totale di esecuzione: $(V + E)\log(n)$. Si tratta dell'implementazione migliore per i grafi sparsi.
- **Liste ordinate:** `extractMin()` usa tempo costante, mentre `reduceKey()` costa $O(V)$. Dunque il tempo totale è di $O(V + VE)$.
- **Liste non ordinate:** `extractMin()` costa V e `reduceKey()` E . La miglior opzione per i grafi completi ha quindi tempo di esecuzione totale $O(V^2 + E)$.

Come già menzionato, Dijkstra non funziona con grafi i cui pesi possono essere anche negativi, perché data la sua proprietà greedy, ovvero che estrarre il nodo con stima

minima garantisce che quella sia già definitiva, dipende dalla non negatività dei pesi. Con un arco negativo, un nodo già estratto dalla coda e inserito nell'insieme potrebbe vedere la sua stima migliorata in seguito, ma non è possibile rivisitarlo.

Per contemplare i cicli negativi possiamo introdurre il secondo algoritmo: **Bellman-Ford**, che consente di trovare la soluzione al problema del cammino minimo da singola sorgente anche in presenza di cicli negativi. Dato un grafo orientato pesato con un nodo sorgente e un peso, restituisce un valore booleano che indica se esiste un ciclo negativo raggiungibile dalla sorgente. In tal caso, ritorna false, indicando che il problema non ha una soluzione; altrimenti ritorna true insieme ai cammini minimi coi relativi pesi.

Algorithm 34 bellman-ford(G, w, s)

Require: G : Grafo orientato pesato, w : Peso, s : Sorgente

 initializeSingleSource(G, s)

for $i \leftarrow 1$ to $|V[G]| - 1$ **do**

for all $(u, v) \in E[G]$ **do**

 relax(u, v, w)

end for

end for

for all $(u, v) \in E[G]$ **do**

if $d[v] > d[u] + w(u, v)$ **then**

return false

end if

end for

return true

▷ Controlla se rimangono archi rilassabili

▷ Se sì, non c'è soluzione

Dopo l'inizializzazione, l'algoritmo effettua un totale di $|V| - 1$ passaggi sugli archi del grafo, ognuna corrispondente ad ogni iterazione del primo ciclo **for** per il rilassamento di ogni arco. Il secondo ciclo **for** controlla l'esistenza di cicli negativi, ed in tal caso ritorna false.

Per confermare la correttezza dell'algoritmo è necessario in primo luogo dimostrare che in assenza di cicli di peso negativo, calcola correttamente i pesi dei cammini minimi per ogni vertice raggiungibile dalla sorgente. Consideriamo un qualunque nodo v raggiungibile dalla sorgente e indichiamo con $p = \langle v_0, v_1, \dots, v_k \rangle$ il cammino minimo aciclico da s a $v = v_k$.

Sicuramente il cammino avrà al più $|V| - 1$ archi, quindi $k \leq |V| - 1$. Ognuna delle iterazioni del ciclo **for** rilassa tutti gli $|E|$ archi. In quanto comprende tutti i nodi all'interno del percorso, è capace di ritornare i cammini minimi per ognuno di loro a partire dalla sorgente. Inoltre, come corollario, abbiamo la sicurezza che esiste un cammino tra due nodi s, v se e solo se l'algoritmo termina con una stima $d[v] < \infty$.

Supponiamo ora per assurdo che l'algoritmo ritorni un valore true anche in presenza di cicli di peso negativo. In tal caso, significa che per ogni arco (u, v) vale la condizione $d[v] \leq d[u] + w(u, v)$. Applicando questa disuguaglianza ad ogni arco del ciclo, otteniamo:

$$\sum_{i=0} d[u_i] \leq \sum_{i=0} d[u_i] + \sum_{i=0} w(u_i, u_{i+1}) \implies 0 \leq \sum_{i=0} w(u_i, u_{i+1})$$

Tuttavia ciò contraddice l'ipotesi che il ciclo sia negativo, perché risulta che sia maggiore o uguale di zero, provando il corretto funzionamento dell'algoritmo di Bellman-Ford. Infine, lo studio della complessità risulta estremamente semplice, poiché è evidente come le operazioni nei due cicli for siano eseguire prima su ogni nodo $O(V)$ e poi su ogni arco $O(E)$, dando come tempo di esecuzione totale $O(V \times E)$.

Un ultimo algoritmo per il calcolo del cammino minimo a sorgente singola è quello di **dagShortestPath()**, il quale sfrutta il fatto che in un grafo orientato aciclico esista sempre un ordinamento topologico. Ciò garantisce che quando si rilassa un arco, la stima $d[u]$ è già quella definitiva, perché nessun arco potrà tornare su u .

Per ipotesi il grafo è aciclico ed è quindi possibile lavorare anche con pesi dal valore negativo. Richiede un'inizializzazione, un ordinamento topologico tramite `topologicalSort()` e un rilassamento degli archi visitando il grafo ordinato. Sfruttando la struttura del dag, ottiene una complessità temporale di $O(V + E)$.

Algorithm 35 `dagShortestPath(G, w, s)`

Require: G : Grafo orientato aciclico, w : Peso, s : Sorgente

`initializeSingleSource(G, s)`
`topologicalSort(G)`

for all $u \in V[G]$ ordinato **do**
 for all $v \in Adj[u]$ **do**
 `relax(u, v, w)`
 end for
end for

4.7 Cammini minimi a sorgente multipla

Floyd-Warshall, Johnson

4.8 Flusso massimo

Ford-Fulkerson, Karp

Capitolo 5

Progettazione e analisi di algoritmi

5.1 Programmazione dinamica

5.2 Programmazione greedy

5.3 Programmazione lineare

Simplex, algoritmo fondamentale polinomiale basato sugli ellissoidi